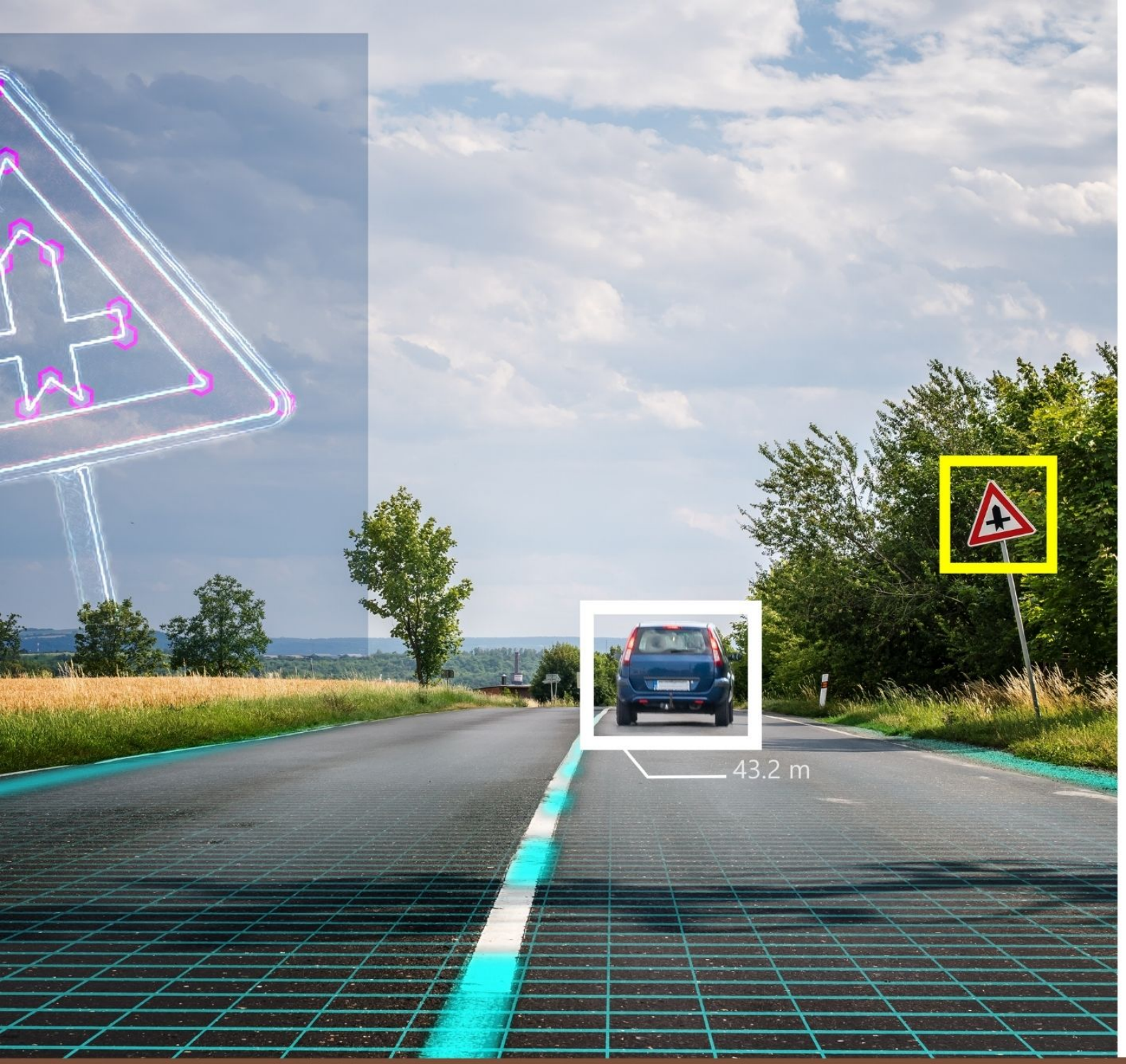


COMPUTER VISION

Prof. Jayanta Mukhopadhyay
Computer Science and Engineering
IIT Kharagpur

INDEX

S. No	Topic	Page No.
	Week 1	
1	Fundamentals of Image Processing Part I	1
2	Fundamentals of Imagr Processing Part II	21
3	Image Transform Part I	39
4	Image Transform Part II	58
	Week 2	
5	Projective Geometry â€“ Part I	79
6	Projective Geometry â€“ Part Ii	94
7	Projective Transformation	114
8	Homography:Properties â€“ Part I	140
9	Homography:Properties â€“ Part Ii	155
10	Homography:Properties â€“ Part Iii	172
	Week 3	
11	Camera Geometry â€“ Part I	189
12	Camera Geometry â€“ Part Ii	208
13	Camera Geometry â€“ Part Iii	223
14	Camera Geometry â€“ Part Iv	240
15	Camera Geometry â€“ Part V	261
	Week 4	
16	Stereo Geometry â€“ Part I	279
17	Stereo Geometry â€“ Part Ii	295
18	Stereo Geometry â€“ Part Iii	310
19	Stereo Geometry â€“ Part Iv	322
	Week 5	
20	Stereo Geometry â€“ Part Vi	334
21	Stereo Geometry â€“ Part V	346
22	Stereo Geometry â€“ Part Vii	363
23	Stereo Geometry â€“ Part Viii	380
	Week 6	
24	Feature Detection And Description â€“ Part I	394
25	Feature Detection And Description â€“ Part II	413
26	Feature Detection And Description â€“ Part III	432
27	Feature Detection And Description â€“ Part IV	449

28	Feature Detection And Description â€“ Part V	474
	Week 7	
29	Feature Matching And Model Fitting- Part I	485
30	Feature Matching And Model Fitting- Part Ii	502
31	Feature Matching And Model Fitting- Part Iii	514
32	Feature Matching And Model Fitting- Part Iv	527
33	Feature Matching And Model Fitting- Part V	540
	Week 8	
34	Color Fundamentals And Processing-Part I	558
35	Color Fundamentals And Processing-Part Ii	574
36	Color Fundamentals And Processing-Part Iii	591
37	Color Fundamentals And Processing-Part Iv	613
38	Color Fundamentals And Processing-Part V	633
39	Color Fundamentals And Processing-Part Vi	651
40	Color Fundamentals And Processing-Part Vii	665
	Week 9	
41	Range Image Processing â€“ Part I	679
42	Range Image Processing â€“ Part Ii	693
43	Range Image Processing â€“ Part Iii	707
44	Range Image Processing â€“ Part IV	720
45	Range Image Processing â€“ Part V	735
	Week 10	
46	Clustering And Classification Â€“ Part I	757
47	Clustering And Classification Â€“ Part Ii	780
48	Clustering And Classification Â€“ Part Iii	799
49	Clustering And Classification Â€“ Part Iv	816
50	Clustering And Classification Â€“ Part V	832
	Week 11	
51	Dimensional Reduction And Sparse Representation â€“ Part	855
52	Dimensional Reduction And Sparse Representation â€“ Part	871
53	Dimensional Reduction And Sparse Representation â€“ Part	891
54	Dimensional Reduction And Sparse Representation â€“ Part	909
	Week 12	
55	Deep Neural Architecture And Applications â€“ Part I	927
56	Deep Neural Architecture And Applications â€“ Part II	950
57	Deep Neural Architecture And Applications â€“ Part III	970

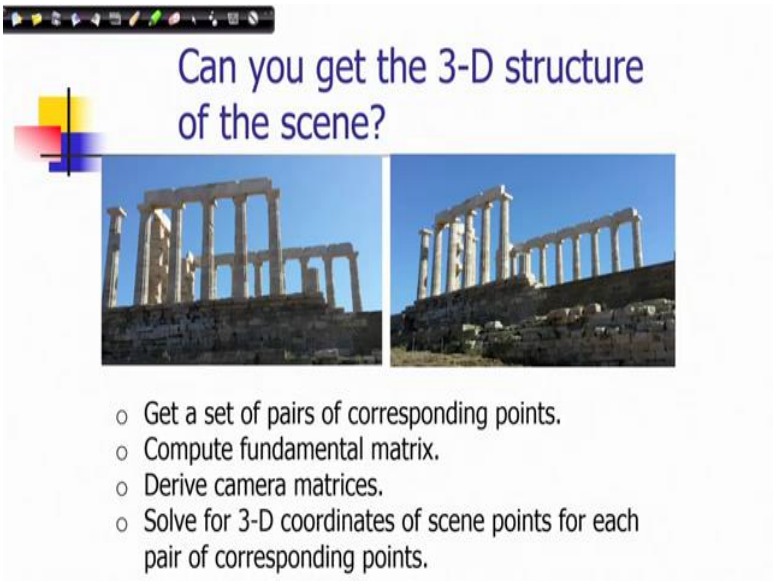
58	Deep Neural Architecture And Applications â€“ Part IV	988
59	Deep Neural Architecture And Applications â€“ Part V	1007
60	Deep Neural Architecture And Applications â€“ Part VI	1028

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 29
Feature Matching and Model Fitting – Part I

In this lecture we will start on a new topic on Feature Matching and Model Fitting.

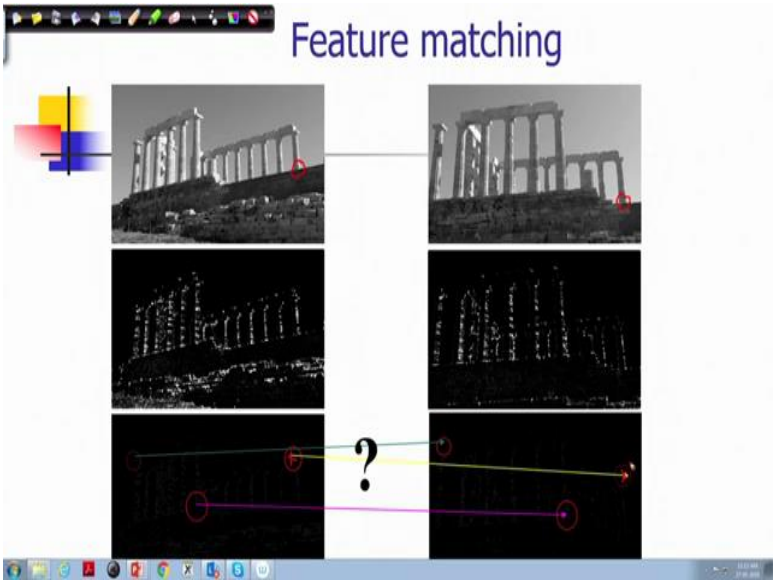
(Refer Slide Time: 00:24)



Let us consider a typical problem where you require this kind of matching of features for example, you would like to compute the 3-D structure of a scene. So, we have already gone through the computational aspects of this particular problem and we can design an algorithm with some computational steps to solve it. So, what we can do? We can get a set of pairs of corresponding points, that is the first thing we should get and in fact, this is a step, where we will see that matching is very much required.

But just to give a holistic view of the solution of this problem. So, you need a set of pairs of corresponding points, then you should compute the fundamental matrix, and then we should derive the camera matrices and then we can solve for 3-D coordinates of scene points for each pair of corresponding points. So, you see that your initial assumption. So, having a set of pairs of corresponding points that itself requires some introspection. So, for in our computations we have assumed they are available either by visually finding out which points corresponds to what. So, let us consider these particular structures.

(Refer Slide Time: 01:51)



So, what we can do at least if you would like to automate this process, we would like to find out first thing interesting points in those images as the feature detection we have discussed. So, we can apply those transformational invariant feature detectors like SIFT detector, Harris corner detectors and also apply the detections in a scale invariant and transformation invariant manner.

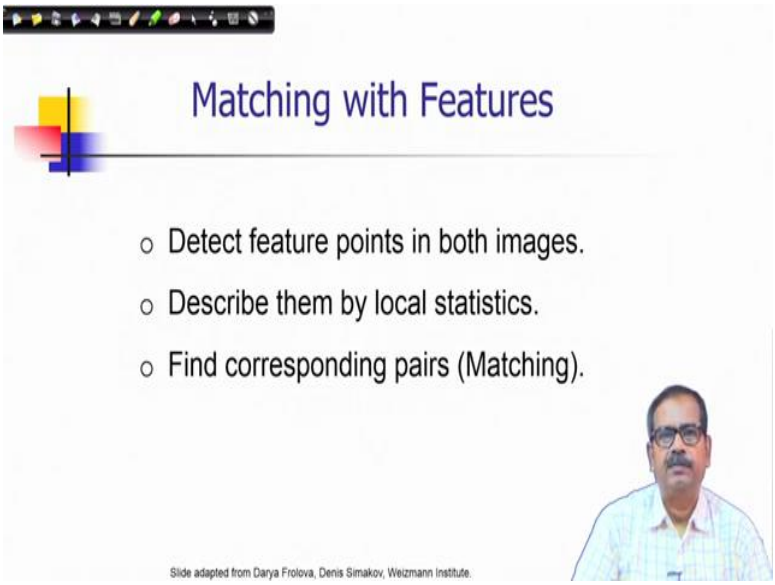
So, consider this particular operations where you define a measure like Harris corner measure and then from there you get the feature locations by finding out the local maxima out of this set of points and those points define your feature points. Similarly for the other view also you perform the similar operations.

So, now since you expect that these points which you have detected their transformation invariant, which means even after the change of view in the imaging still those points will be retained in your detect after detections. So, we could try to find out that what are the correspondences or which point corresponds to which point of the other image, which means that we need to find out this kind of relationships where a point in the left image corresponds to the image another point in the right image and their relation is that they are images of the same point. So, you can check with this particular thing.

So, we are trying to find out for example, by precisely locating the feature points we are trying to get a correspondence and in this way you would like to get the set of corresponding points. So, computation which involves in pairing these points or getting

the corresponding pairs of corresponding points that is what is the what is known as feature matching. So, we will be discussing some of these techniques about this feature match.

(Refer Slide Time: 04:27)

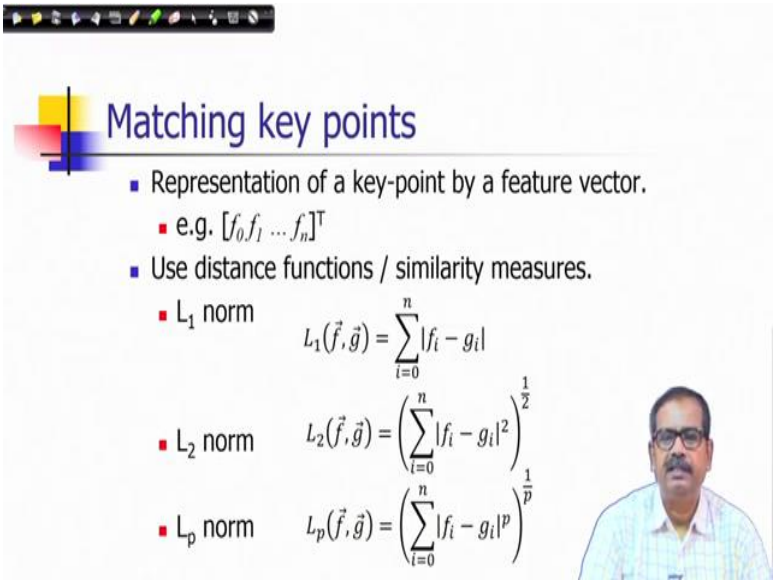


So, this is the summary of that computations that we discussed that first you have to detect feature points in both images and then you should describe them by local statistics. So, this is the way how these points are to be you know matched that just their locations will not give any particular information about the nature of the point.

So, you need to at least look at the neighboring statistics neighboring distribution of intensity values or some functional you know distributions in it is particular neighborhood and which you are expecting that would also remain the similar in the other image of the similar of the same landmark point around that same key point and then exploiting that similarity we are trying to match them.

So, you have to describe them by local statistics and we have discussed about different feature description techniques like shift descriptors or what we different kinds of descriptors and which have the properties of these transformation ingredients and then you have to find corresponding pairs. So, we will be considering that what competition involves suppose we are given those descriptions, then how do you say that this pairs this pair of points are the corresponding points?

(Refer Slide Time: 05:53)



So, we have already discussed in the previous topic also because while explaining the motivations of describing a key point that one of the computations that we consider that matching is in this representation. So, same description or same discussions will be repeating here that, you can represent a key point by a feature vector that you have considered. That suppose we have a feature representation of a vector here it is an n plus 1 dimensional vector $[f_0, f_1 \ldots f_n]^T$. And then you can use some distance function to define the proximity or similarity between two vectors, you can other than distance function also directly you can use any similarity measures also.

So, some of the examples of this distance function are given here like you can use L1 norm,

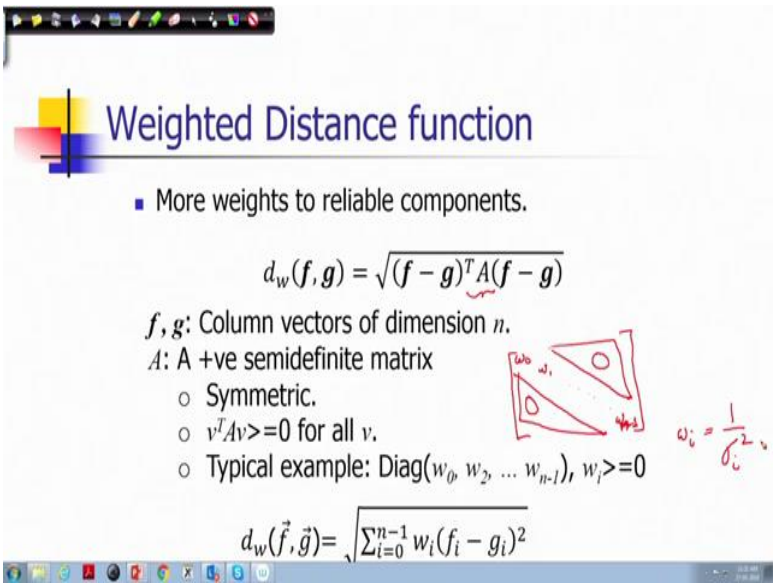
$$L_1(\vec{f},\vec{g}) = \sum_{i=0}^{n} |f_i - g_i|$$

$$L_2(\vec{f},\vec{g}) = (\sum_{i=0}^{n} |f_i - g_i|^2)^{1/2}$$

$$L_p(\vec{f},\vec{g}) = (\sum_{i=0}^{n} |f_i - g_i|^p)^{1/p}$$

So, you can use these distance functions to define the proximities between key points and you can apply certain strategies we will discuss that later on that how to declare that two key points, they correspond to the same scene point for any way for two images.

(Refer Slide Time: 08:21)



Sometimes no distance functions you can have weighted distance functions which means in the in our previous examples of distance functions, we considered every component has uniform weight we did not distinguish that which component is more reliable and which component is less reliable. So, if it is more reliable we give more weight to that differences if it is less contributing to the discrimination of the images reliable means this it is contribution to the discrimination between two images.

$$d_w(f,g) = \sqrt{(f-g)^T A(f-g)}$$

So, if it is less then we give placement. So, this can be represented in this particular mathematical form as you can see that, it is a it is used the column vector representation of a feature vector and using matrix operations, you can compute this distance. So, in this expression particularly note that A is a positive semidefinite matrix and which means that it is a symmetric matrix and if I perform this operation

$$v^T Av \geq 0, for\ all\ v$$

So, its value should be positive with respect to the same vector and typical example of A is given here. So, that this is a diagonal matrix which we can represent in this form that you can write

$$Diag(w_0, w_2, \ldots w_{n-1}), w_i \geq 0$$

So, w 0 today in the previous example we have the index from 0 to n which was in n+1 dimension space. So, this is the diagonal matrix and all others are 0. So, this part and this part they are all 0. So, if I multiply if I use this diagonal matrix here in this expression then it is simply this expression will boil down to in this form

$$d_w(\vec{f}, \vec{g}) = \sqrt{\sum_{i=0}^{n-1} w_i(f_i - g_i)^2}$$

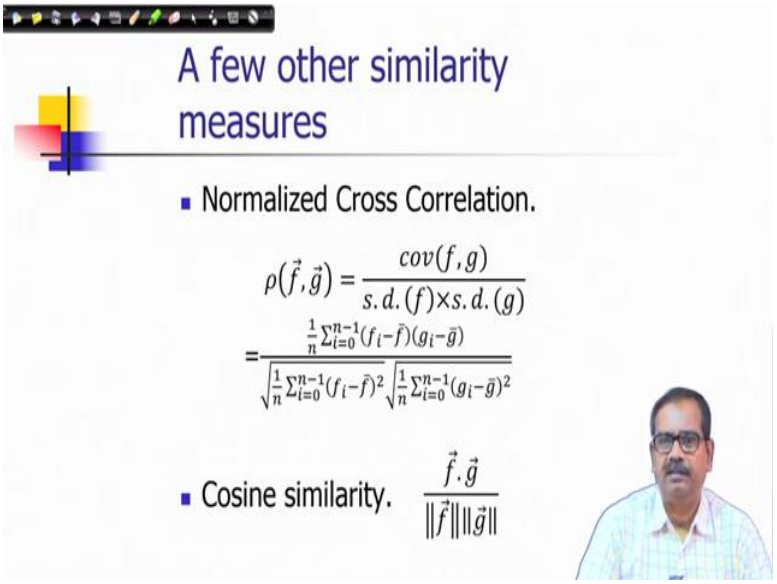
So, one typical example could be that they could be weighted by inverse of the standard deviations of the variations in that ith component of the feature vectors say we can consider or variants rather.

So, if I consider

$$w_i = \frac{1}{\sigma_i^2}$$

Where σ_i is the standard deviation of the ith component of this feature vectors you consider all possible feature vectors. So, what is the variability on the ith component and then if it if it has more variability we give less weight if it has less variability we give more weight to define this distance. So, this could be one such policy while defining this distances one typical example of weighted distance function when we can use $w_i = \frac{1}{\sigma_i^2}$ and σ_i is the standard deviation of the ith component if I consider the statistical distribution of the ith component of these vectors.

(Refer Slide Time: 12:19)



There could be other kinds of similarity measures. So, instead of distances by distance compute the distance between two feature vectors means smaller the distance bit greater is a chance of having them or declaring them similar or the smallest distance between a pairs of feature vector may indicate that they are the matching pairs whereas, for similarity it is just the inverse relationship.

Where in a similarity measure the higher the similarity value higher is the chance that they are matching candidates. So, two such similarity measures which are used here one is called normalized cross correlation measure and the other one is cosine similarity. So, let us consider their mathematical forms.

So, this is the normalized cross correlation measure which you can see that it is defined as the correlation between no two distributions where here distributions are considered in terms of their components. So,

$$\rho(\vec{f},\vec{g}) = \frac{cov(f,g)}{s.d.(f)Xs.d.(g)}$$

$$= \frac{\frac{1}{n}\sum_{i=0}^{n-1}(f_i-\bar{f})(g_i-\bar{g})}{\sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(f_i-\bar{f})^2}\sqrt{\frac{1}{n}\sum_{i=0}^{n-1}(g_i-\bar{g})^2}}$$

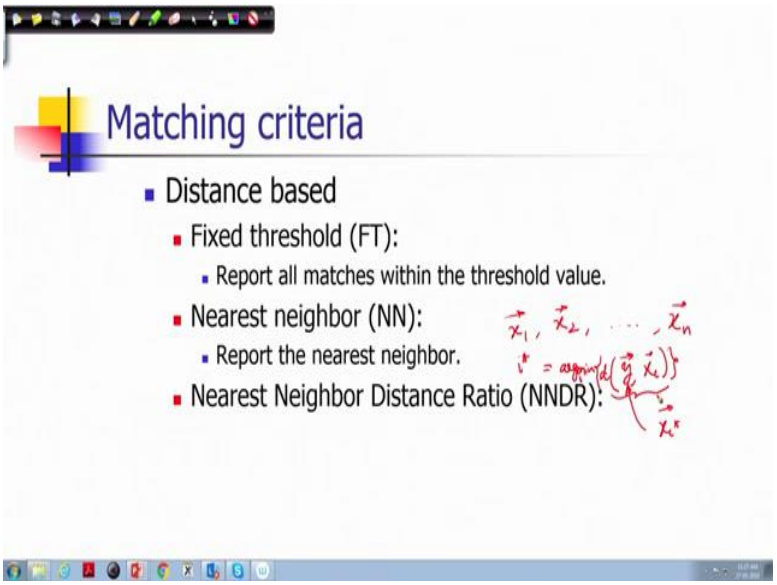
They have been expanded also in my definitions and as I mentioned that, here we are considering the values of components across the vectors and we are comparing those variabilities with respect to two different vectors. And if they are very similar this group cross correlation value should be you know high actually it ranges from [-1, 1].

So, if they are very highly similar then it should be close to one whereas, the other function which is called cosine similarity that is defined in this way.

$$Cosine\ similarity = \frac{\vec{f}.\vec{g}}{\left|\left|\vec{f}\right|\right| ||\vec{g}||}$$

So, as you understand here also, if the vectors are very similar these angles should be near to 0. So, which means the cosine of that angle 0 should be equal to 1. So, here also the values range from [-1, 1] and the values which are nearby and which are very similar there the value should be high, it should be close to 1.

(Refer Slide Time: 15:09)



So, let us now consider what kind of different matching criteria could be there. So, I will consider distance based matching criteria mostly here and you can extend this discussion or extend these ideas for using similarity based matching. So, in the distance based matching the minimum is the distance or smaller is the distance better is the you know matching between two vectors, that is a way that this policy is considered. So, one of the

policy of distance based matching is that, you can use a fixed threshold which means you can report all matches within that threshold value.

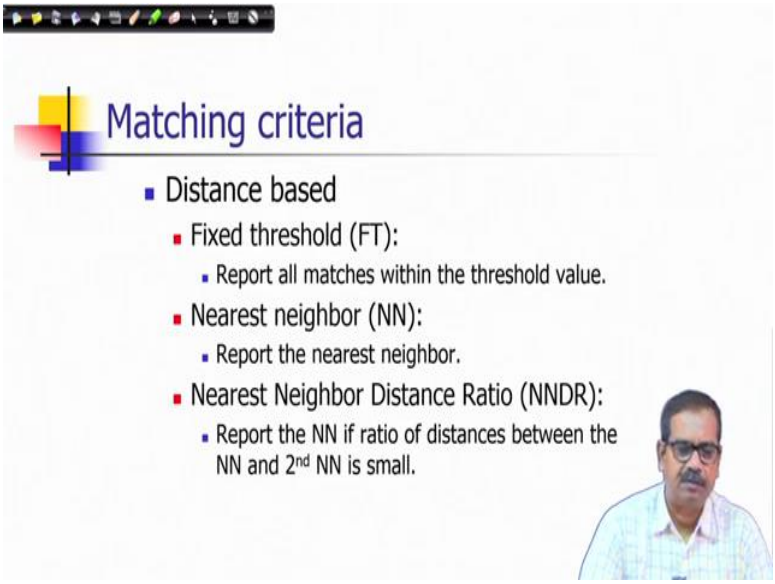
So, it is not declaring just a pair of feature vectors, but given a feature vector it is providing you a candidate feature vectors and then you may have to do some post processing to select which one is the fittest or which one the actual feature vector is corresponding to the query feature. Similarly, but if you would like to precisely define the corresponding feature vector, the nearest neighbor definition is much more useful because here it is not a fixed threshold.

So, it considers which feature vector is nearest to it. The problem is that you need to also consider; that means, it does not consider that whether the the distance between them is greater than certain threshold value or within certain you can combine them of course, but by definition nearest neighbor is the feature vector corresponding to that.

So, if I say that there are feature vectors say [x1, x2, …, xn] those are the candidate feature vectors and you have a feature vector y in another image and this is the feature vectors in the other image. So, what you are doing? You are computing distance between this each feature vectors and which one is the smallest out of them. So, you take the minimum of them that operation called arg min and that would give you the corresponding feature vector; that means, xi* that is a nearest neighbor of y.

But it may happen that the distance is greater than the fixed threshold what you have considered. So, you can apply that consideration also while selecting this nearest neighbor point. So, this is the strategy when you are considering nearest neighbor principle of detecting or matching a pair of feature vectors.

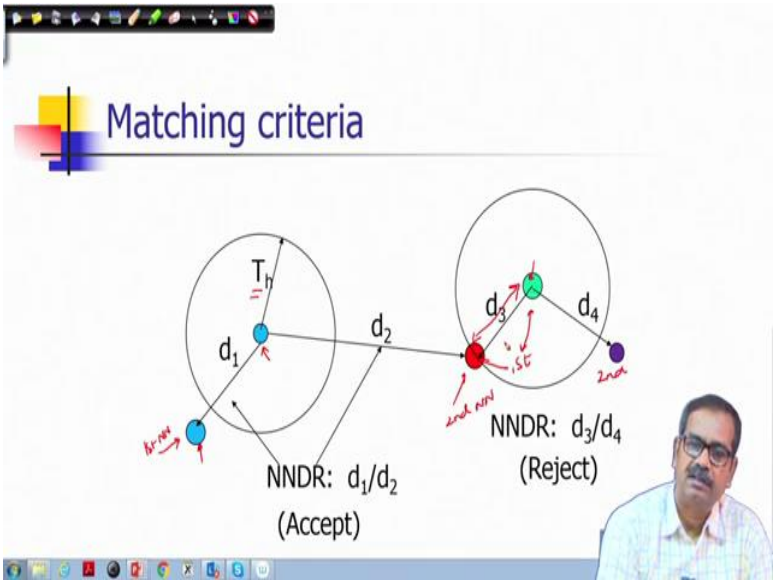
(Refer Slide Time: 18:22)



The other strategy which is found to be more robust is called nearest neighbor distance ratio. So, in this case it considers the ratio of distances between the nearest neighbor and the second nearest neighbor. So, that means, if it is distinctly nearest, the next neighbor is quite far away then we should accept that as a reliable matching between these two feature vectors. So, which means this ratio should be very small it is a distance between the nearest neighbor and the second nearest neighbor.

So, ratio should be very small and if this ratio is very small then you can consider that this is a good matching point. Here also we are not giving an absolute criteria it is a relative it is relative distances what we are comparing. So, relative comparison between two distances one with the one distance is with the nearest neighbor and the other one is with the second nearest neighbor.

(Refer Slide Time: 19:26)



So, there is a figure by which you can understand this particular process. So, in this example you can see that the feature vectors which are actually closer in the space they are colored by the same color. So, for example, if I use a fixed threshold of th, then all the feature vectors within these circular region around it. So, here we are showing a two dimensional space.

So, it could be sphere, if it is a three dimensional feature time feature vector or for an n dimensional feature vector it would be a hyper sphere. So, anyway. So, within this region if there is any feature vector that would have been reported, but what happened the nearest neighbor of this one is just outside of the circle. So, if you use fixed threshold you will be missing this.

If we use nearest neighbor principle then this will be also selected which means you get a true match according to this particular configuration and also it matches with the ground truth as you have considered that they are the two matching points closest neighbor. But if I consider nearest neighbor principle then what happens you see that for this point this is the nearest neighbor.

But it is not the corresponding matching point it is showing with the different color in this particular case. So, though it is nearest neighbor, but still it should not be considered. So, that they are the nearest neighbor distance rule comes if I consider the nearest neighbor distance rule then with respect to this particular point, this is the first nearest neighbor

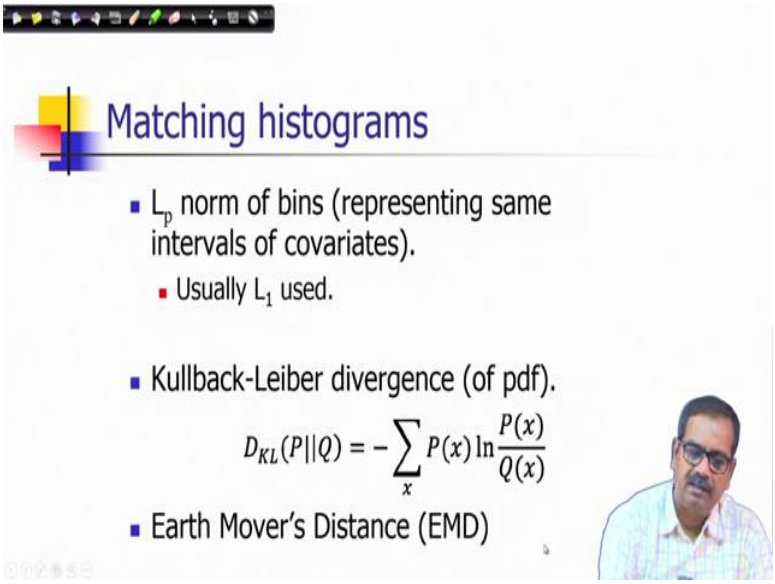
which means that this is the nearest neighbor and this is the second nearest neighbor with respect to this point.

So, if I take the ratio of between these two and this ratio is expected to be small according to this diagram and then also we accept it. Which means that even the principle of nearest neighbor distance ratio which is shown here in acronym NNDR, even if you apply them then also it is accepted whereas, if I consider the other situation you can see that here this the with respect to this image this point this one is the first nearest neighbor with respect to this one and this is a second nearest neighbor.

And the second nearest neighbor is also quite close as you can see, because no there is an ambiguity of descriptors in this case. And if I take the ratio d3/d4 then this ratio is expected to be higher and then with the appropriate thresholding you can consider and you can reject them.

So, this policy in NNDR policy it is also rejecting in this case and which is desirable, because the using nearest neighbor principle this will be assigned this list will be matched using nearest neighbor principle, but which is not desirable according to the given data. So, that is why policy of nearest neighbor using distance ratio is found to be more robust than the other policies.

(Refer Slide Time: 23:12)



So, we discussed about matching of key points here using their feature descriptors, what about regions or even the images where you represent a feature description a feature where you have a feature descriptor in the form of histograms of certain measurable quantity so, there you require matching of histograms.

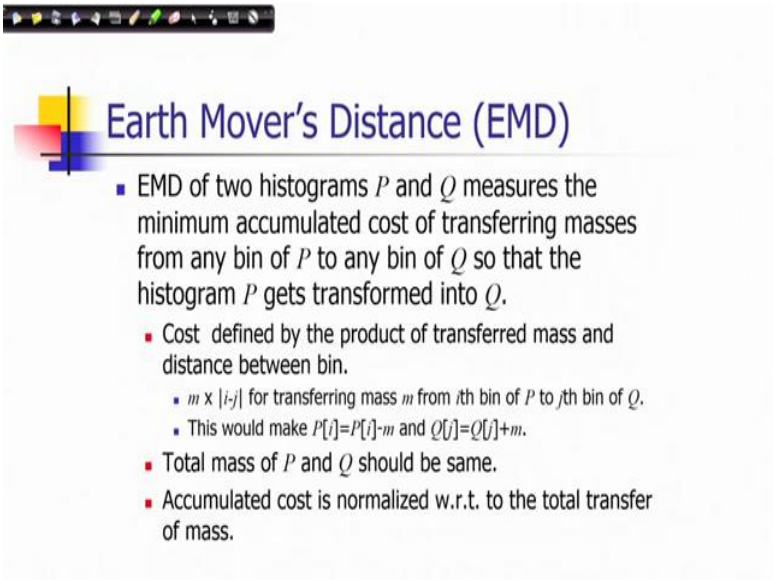
Now histograms also can be considered as a feature vector each bin represents as a component of a feature vector. So, there you can use usual distance functions what we discussed in the previous case also like Lp norms could be used and with respect to the corresponding bin, and usually L1 norm is used to mostly for representing this histograms that has been found, but you can use any other norm. But there are others special measures other special distance functions or measures by which know you can describe the differences between two histograms or similarities between two histograms.

So, one of this measure is called Kullback Leiber divergence measure and it actually it tells you if I give you two probability density functions probability distributions, two different probability distribution how close or how different they are. So, the measure is defined in this way;

$$D_{KL}(P||Q) = -\sum_{x} P(x) \ln(\frac{P(x)}{Q(x)})$$

There is also another distance function which is called earth movers distance function and which I will elaborate a bit more.

(Refer Slide Time: 25:16)



So, in this case the idea is that you have two histograms P and Q and you consider that transforming one histogram P to Q by transferring masses from a bin to any other bin of Q. So, you are assuming that every bin represents every bin has certain amount that is a definition of histogram it is a frequency distribution. So, frequency is considered frequency of that particular quantity that itself each unit is called as a mass.

And if you are going to transfer a portion of that to another bin and of a Q and so, that the histogram P gets transformed into Q. So, this transfer has a cost and this cost could be defined as the product of transferred mass and distance between bin which mathematically I can express in this form that m and the distance between two bins.

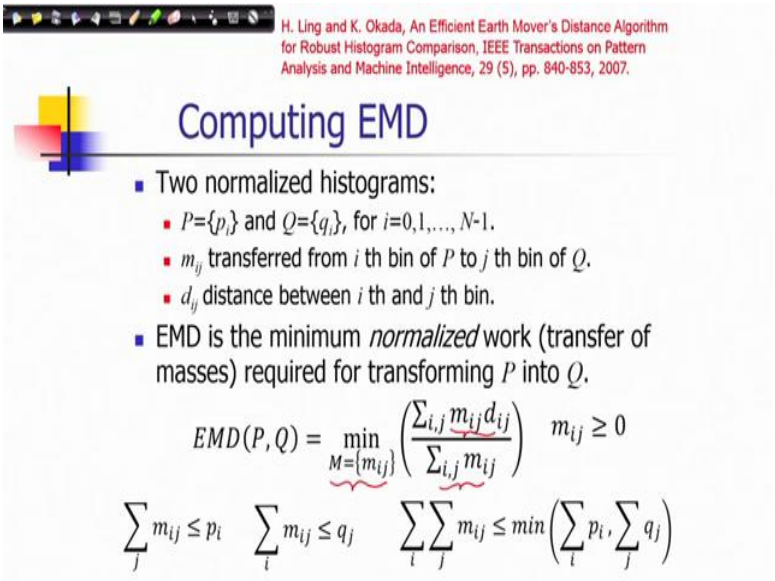
So, each bin say ith bin and jth bin their difference of the bin locations itself could be considered as a distance that absolute differences between bin locations. So, you take the product of these two that is what is the mass you are transferring from from ith bin to jth bin. So, for example,

$$P[i] = P[i] - m \; and \; Q[j] = Q[j] + m$$

So, this is the transfer operation that would do and by doing this thing what you are trying to do trying to achieve is that transform histogram P into Q. So, we consider in this context the total mass of P and Q should be same.

So, after doing all these transfers you are expecting that no you can convert the distribution of masses in P in the form of P should we now in the form of Q. So, what is the minimum cost operation accumulated cost operation? So, particularly this accumulated cost again it could be it should be normalized with respect to the total transfer of mass that is a measure. So, which what is that minimum no cost and that will give you the distance and that is what is called earth movers distance, because no it relates like transferring mass is something like digging the earth from one part from one place and placing it to the that amount to the another bin.

(Refer Slide Time: 28:26)



So, something from that analogy the name has come and providing you a bit more mathematical formulation of this computational problem, that now you consider a two normalized histograms because no as I assume that their mass should be same. So, the safest way to start with is that normalize both the histograms. So, their some of areas should be equal to 1 that is the total mass they have and represent say ith bin of histogram P's small pi and ith bin of histogram Q's small q and now you consider the ranges then there are n number of bins and the mij denotes the mass transform from ith bin of p to jth bin of q and dij distance is expresses a distance between ith and jth bin there is a distance between locations.

So, the earth movers distance is the minimum normalized work required to transforming required for transforming p into q. So, which means you are trying to compute this one.

So, this is an optimization problem that you have to minimize this particular quantity as you can see this is the cost of transfer from ith bin to jth bin of a mass mij and this is a total mass that has been transformed that is the normalization part of the work.

$$EMD(P,Q) = min_{M=\{m_{ij}\}}\left(\frac{\sum_{i,j} m_{ij}d_{ij}}{\sum_{i,j} m_{ij}}\right), m_{ij} \geq 0$$

So, what is the set of transfers that would gives you the distribution Q from P and what is out of those possible sets which one will give you the minimum cost.

So, this is what is your computational problem and since now you are dealing with mass there are certain constraints you need to consider one other constraint is that, every mass should be positive or 0. So, if there is a transfer it has to be a positive transfer then the other one are your transfer of mass from the ith bin of p should should not exceed the content what it has in that bin; that means, that is a capacity that bin has to transfer masses from that to any jth bin of Q. So, so that is expressed mathematically in this form if I accumulate all masses from ith bin, it should not exceed the original mass of that bin in the histogram p.

Similarly then what you are transferring to the jth bin is that should not again exceed what is the content in that jth bin because that is what would like to transform pth P histogram to Q histogram. So, you have to ensure that that that it should go always less than equals it should not exceed that. And in overall the transform or of all these masses should be less than the total mass of this histograms one of this histograms. So, minimum of these two histograms. So, it is actually considering this framework does not really require that two histograms should have same mass, but in our context we considered that they should have same mass.

So, in this case it should be always less than equal to 1 because that is what we have assumed that total mass is 1 in the normalized histogram. So, this is a computational problem and I am not going to discuss this computational solution of this problem which is not within the scope of this particular course, but if you are interested, you can go through this particular paper where a typical solution has been presented and efficient solution this paper has been published in IEEE transactions on pattern analysis and machine intelligence in 2007 and you can find out an algorithm for that. So, with this let

me stop this lecture at this point and we will continue this discussion of matching of feature descriptors as well as the other topic of model fitting in this series of lectures.

Thank you very much.

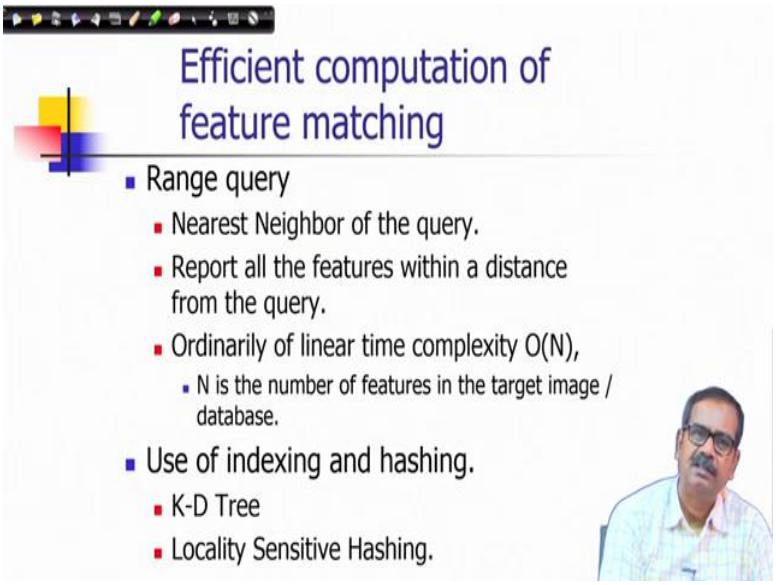
Keywords: Feature matching, nearest neighbor, earth mover’s distance, matching criteria

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 30
Feature Matching and Model Fitting Part - II

We are discussing about Feature Matching and now we will be considering that how this computation could be made more efficient.

(Refer Slide Time: 00:25)



So, we have seen in the case of feature matching, we required to find out similarities between two feature vectors or you need to compute the distances between a pair of feature vectors. And the nature of this particular computation is that if I give you a feature vector for which you have to find out its corresponding feature vector in a candidate set. It is the kind of range query which means instead of a single feature vector I may get a possible candidate's in its proximity that is the kind of answer I may like to get.

And later on I would like to find out which one of them is more closer or satisfies some other criteria by which I can declare them as the close match. Now so, this is a kind of computational problem. So, it is the nature of you know query. It is not typically I would be interested on a particular feature. I may be interested on a set of features or on a range of or an interval on which this feature vectors should like. A special case is of course know you would be only considering all the nearest neighbor of the query and otherwise you

may as I mentioned for a range query, you can report all the features within a distance from the query as we have seen also that is one of the strategy where you can use a fixed distance threshold. And report all features which are within that region as centering around that query feature vector.

But the question is that now when you are performing this computation, you have to compute the distances with all the feature vectors in that in your candidate set that, so if there are n such feature vectors you have to compute n times those distances. So, that makes your operation as a, you know linear time complexity of order. Ordinarily if I apply this brute force method of computing distances to each of them, then it would take off order in time complexity.

So, our objective is that how we can make this you know computations more efficient. As we have already; you might have known that typically this kind of search problem. If you have a set of you know candidate vectors or candidate elements, then you can keep them in some organized manner, so that your search could be made more efficient and you can perform in sub-linear time complexities in many cases. So, we call actually these operations of organizing the candidate set in a particular form that is that operation is also called as indexing with respect to databases particularly. You might have heard this term.

So, in this case n is a number of features in the target image or database, and that is why to make it efficient as I mentioned that you can use some techniques like similar to indexing. Even you can use techniques like hashing where you can store your candidate set by generating some hash values and based on that hash values, the sets having the same hash values would be kept together.

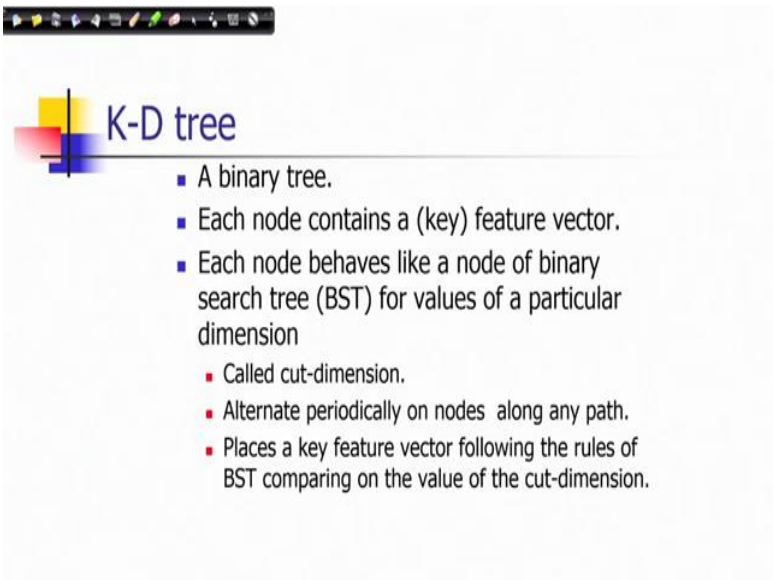
So, your idea is that for a query image it is expected that it should have a similar hash value and it should go to that particular group where your candidate set is expected to be there, and then you can compute the distances of similarities between them. In this particular context there are two techniques which are very popular for this kind of feature matching.

Because you can see that this feature vectors they are multi-dimensional representation and you can use a concept called K-Dimensional tree, K-D tree where K is the known dimension that is representing of the future vector. So if the feature vector is of two dimension we call it 2D tree, if it is three dimensional it is 3D tree, if it is one dimension

it is 1D tree which is which happens to be binary search tree. So, K-D Tree is a is the concept which is extended from binary search tree and that concept is used for searching over a multi-dimensional feature representation. And we will have some discussion, the principal of representation using K-D tree and also the search that we perform over K-D tree. And for hashing there is a technique called Locality Sensitive Hashing, as you can see that in this case the Geometry is also important.

Because know you want to get feature vectors in certain locality of your query feature vector. So, there the geometry concept of geometry space those are to be considered. In a general hashing, it is simply it is a matching of a random point my random element. So, you are randomly you are grouping different elements by the same hash value, but here we need to also consider that the in the group the corresponding candidate sets should have similar, no they should be closely spaced, they should have neighboring properties. So, to ensure that property there is a kind of; there is a technique which is very popular and this technique is locality sensitive hashing and we will also discuss about this technique.

(Refer Slide Time: 06:45)



So, let us consider first the principles behind the K-D tree, as I mentioned K-D Tree is a kind of extension of concept of binary search tree for multi-dimensional feature search. So, it is effectively it is a binary tree and every node it contains a key feature vector like what we have what we know about binary search tree. So, every node contains a key feature vector instead of a single key value for a one-dimensional tree, one dimensional no

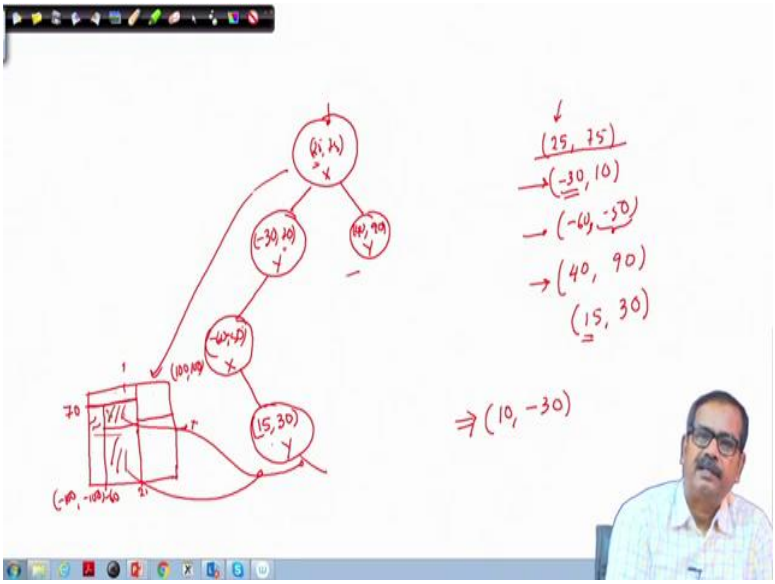
feature representations using binary search tree. And then as I mentioned each node behaves like a node of binary search tree for values of a particular dimension.

So, that is predefined for that position of the node say the, you are starting from the root say root, in the root we will be using only one dimension. Suppose you have a two dimensional feature representation. So, if I consider the dimension 1 is x and dimension 2 is y.

If I represent those dimensions, then say root will be we will consider the you know the rule that would be used for partitioning the values or partitioning the space based on only one dimension same of dimension x. I will explain it in the subsequently. So, we call this dimension as cut dimension. Just referred to this that dimension based on which this partitioning takes place and it alternate periodically on nodes along any path. So, that is how a key value is placed.

So, like binary search tree is the order of values by which a tree structure is determined, here also as and when the feature vectors are coming you are keeping them in to in this structure and when you are expanding a note, when you are placing it you need to understand, find out that you know which dimension to be used for placing it into a particular sub-tree or in a particular node. So, that is what places a key feature vector following the rules of binary search tree comparing on the value of the cut dimension. Let me explain the formation of a K-D tree.

(Refer Slide Time: 09:29)



Let us consider a set of values say we assume that we have say values say 25, 75; we assume your feature dimension is a two-dimensional feature vector. Some values let us consider say minus 30, 10 minus 60, minus 50 40, 90 take it as 15, 30. So, I am assuming that I have five feature vectors forming a candidate set. So, I have to organize this feature vectors into a K-D tree one by one I have to insert an element. So, I will start from the root and in the root I will be considering the dimension x so, this element would be inserted in the root and the dimension. So based on these value only my next so, here the x-dimension should be compared when we consider insertion of the, so insertion of the next element.

So, the next value is minus 30, 10 so, as minus 30 is less than, so this is the dimension we will be considering here. So, minus 30 is less than 25 according to the binary search tree rule so, I have to place it here. And next is minus 60 minus 50 so, if I go here, so first I have to compare minus 60 and since this is this dimension x, so minus 60 it should go there.

Now, minus 50 would be compared because now the y-dimension in this node, the y dimension has to be compared. So, minus 50 is less than 70 so, it should go in this node. So, this is minus 60 minus 50, here y-dimension is compared; consider so, in this case we need to compare the x-dimension, right. So, consider the next element so 40.

So, first we compared from the root 40; so, 40 means it should come to this place so, this is vacant. So, it will be placed here itself 40, 90 and this node the cut dimension y to compare, next is 15, 30 so, if I come here, so 15 it will be compared with 15 so, it will be coming to this node. Now it will be compared with 30, it will be coming to this node because 30 is less than 70. So, now again the x will be compared, it is 15; 15 is more than minus 60. So, this value is 15, 30 and here the cut dimension is y. So, in this way as you can see a tree is built. So, as you go on including more number of elements your tree will be increasing its levels and also it will be expanding its nodes.

And in this particular so, any query image suppose you have you have a query say 10 minus 3 and we would like to get the nearest neighbor from this query. So, what we can do that it is not simply like you can apply the binary search tree rule because even if you move to certain sub-tree, the other sub-tree also may contain the nearest neighbor.

Because if I note how the partitions of this space is made in this fashion, say let me consider a space where you have if the values are ranging from minus 100, minus 100 to say 100,

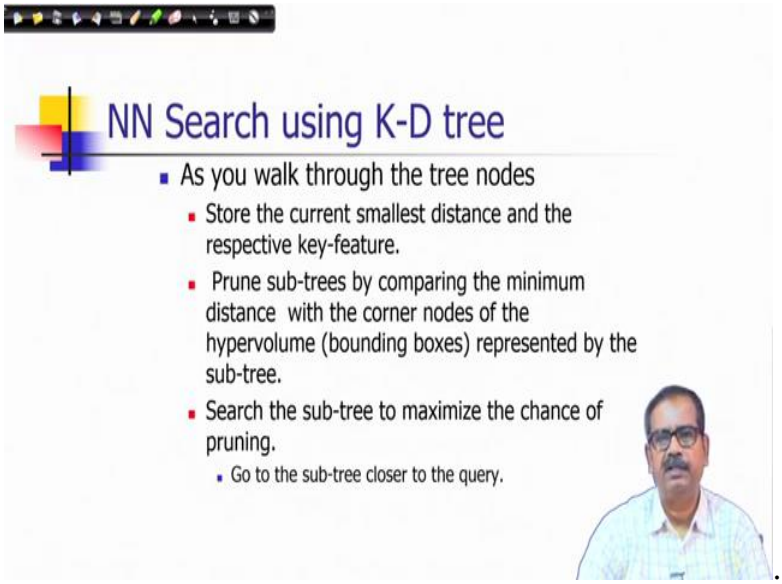
100. And in the first case as you can see that when we are in the root, it is 25 sets. So, you are using 25 so, it is minus 100 is this so, 25 maybe this location for the x-dimension so this is x, this is y.

So, all the values which are more than 25, so this is a subject of the space at the root location; from the root location. So, all the values; all the points two dimensional points which are which will be in the sub-tree should lie in this particular subspace, a box and all other values, all these sub-trees that should lie in this space. So, that is idea and then you consider minus 30, 70 and that is in this case you have to use the y coordinate. So, 70 would be used for; so, you are now in this book sub-tree left sub-tree so, 70 could be somewhere here. So, this is a region which will be representing this particular sub-tree minus 30, 70; so, this is 70 so, this is 25 x direction and say this is 70, if I consider 40, 90 so, that should be also in the y region rather 40, 90 is a point. So, these are the two regions.

So, this should be the left sub-tree of this one and this should be the you know right sub-tree. Similarly this partition, this is the left partition of this node and this is the right partitions. So, when you have minus 60 minus 50 at as it is in the left notation, so you should consider here and it is the dimension x. So, it will be something like this. So, this is minus 60; one second this is a left and this is right and when it is 15, 30 that is from the y, so it should be something like here. So, you can see that for this node, your this node is the right sub-tree and this node is the left sub-tree.

So, in this way you are subdividing a region into different cells and your query point should be expected your if you would like to place a point, it should be placed on in that region and it is expected that your search space, your neighboring point should be in that region. But the problem is that some of the points even in this bordering region they could be also neighbors. So it is not sufficient to search just this cell, you have to search also the neighboring searches. So, you have to follow certain search strategy which is not exactly the same as binary search strategy to find the nearest neighbor. So, let us discuss that also here.

(Refer Slide Time: 18:25)



So, we will be considering a nearest search using K-D tree as we have seen that how K-D tree partitions the space and trying to locate a query into a particular cell, though its neighboring cells are also to be considered for finding the neighboring regions. So, the strategy is that as you walk through the tree nodes, you should always know the current smallest distance and the respective key feature.

You should not ignore any sub-tree when while traversing this node for searching a feature, key feature, tou should not ignore it unless you make sure that all the values, all the feature vectors or all the key vectors within that sub-tree they are at a distance which is greater than the minimum distance what has been found so far among the features space. So, that is why you have to store the current smallest distance and the respective key feature, then you can prune sub-trees by comparing the minimum distance with the corner nodes of the hyper volume, the here the bounding boxes what you have shown.

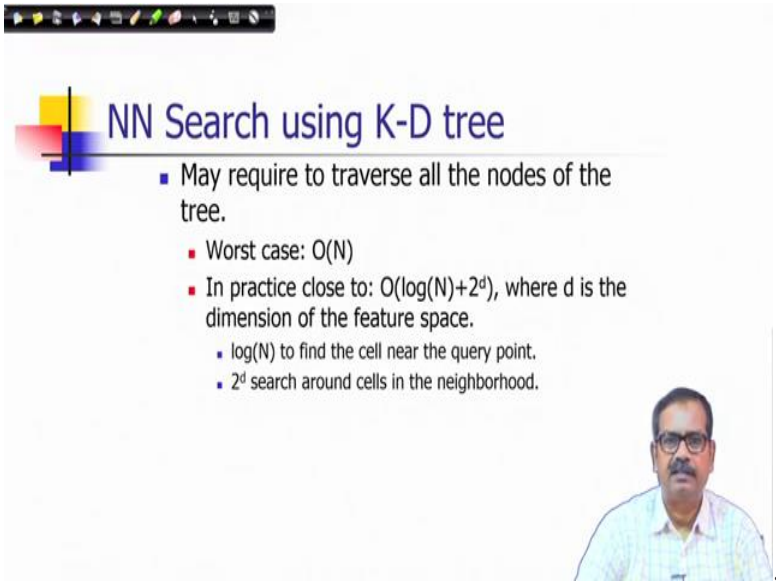
So, you can also note down their corner points and you can compare them these distances. If you find that one of these corners, see all the distances of this corner points they are greater than the minimum distance, then you need not go through. You should not go through that searching of any key values in that region. So, in this way you can prove.

So, while taking a decision in the sub-tree where you should move next, you should search the sub-tree to maximize the chance of pruning which means go to the sub-tree which is closer to the query. So, you are not no, you are not you are not immediately taking any

decision of removing any sub-tree from your search space as we do in the binary search tree., at every note we take a decision and we go either left or right, but in a K-D tree we cannot do that.

We have may traverse along both the sub-trees unless you make sure that the sub-tree, the cell represented by the sub-tree, all the corner points they are at the distance greater than the smallest distance what you have in your current execution and then you can prune that. So, in this way you go and finally, whatever sub-trees you are having at the end you have to compare all the feature vectors in those sub-trees to find out the neighboring vectors.

(Refer Slide Time: 21:11)

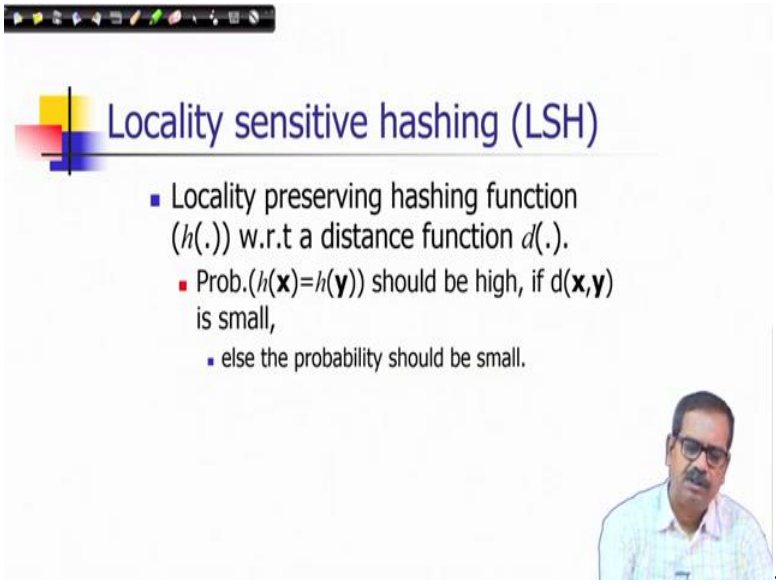


So, the time complexity if we consider like even in binary search tree also worst case space time complexity, still linear in K-D Tree also it would be linear, but in practice you may get

$$O(\log(N) + 2^d), where\ d\ is\ the\ dimension\ of\ the\ feature\ space$$

So, log(N) to find the cell near the query point and 2^d for searching around the cells in the neighborhood. As I mentioned that it is just not that cell, you have to consider also neighboring cells.

(Refer Slide Time: 21:45)

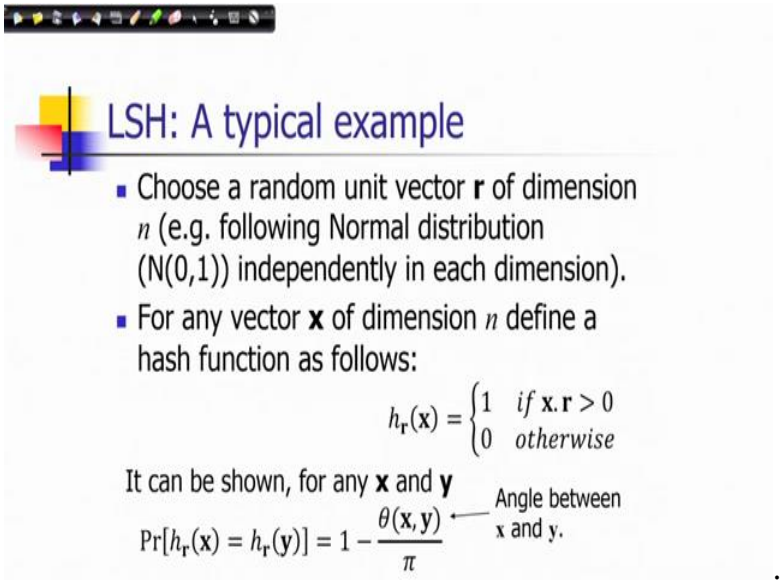


The other technique for performing this efficient search is locality sensitive hashing technique and I would also elaborate a bit more about this technique. I will provide you the principles without going into much details. So, the property here is that you have to use an hash function which should preserve the locality of feature vectors with respect to distance function which means that if I consider a hash function say h, and if I consider two feature vectors x and y which are being shown here by bold fonts.

So, the both the hash functions if they are same, so their probability that both of them would be same would be very high. If the distance is between them is also small, that means they are very closely spaced. So, if I can ensure this property of a hash function, then those hash functions will be useful in placing similar features into the same group, same bucket.

So, that is a principle we will be using. So, let us find; let us consider an example of a hash function as it is true that know this should be the probability should be high when their distances are small. Similarly probability of hash function should be different when the distances are large. So, it should satisfy that property also.

(Refer Slide Time: 23:19)



So, a typical example as we can consider that, consider a random unit vector r of dimension small n. And for example, you can form this vector from using a normal distribution independently generating the values following this distribution in each dimension. That would give you n times if you do, then that would give you a feature vector of random feature vector of dimension n. So for any vector x of dimension n, you can define a hash function.

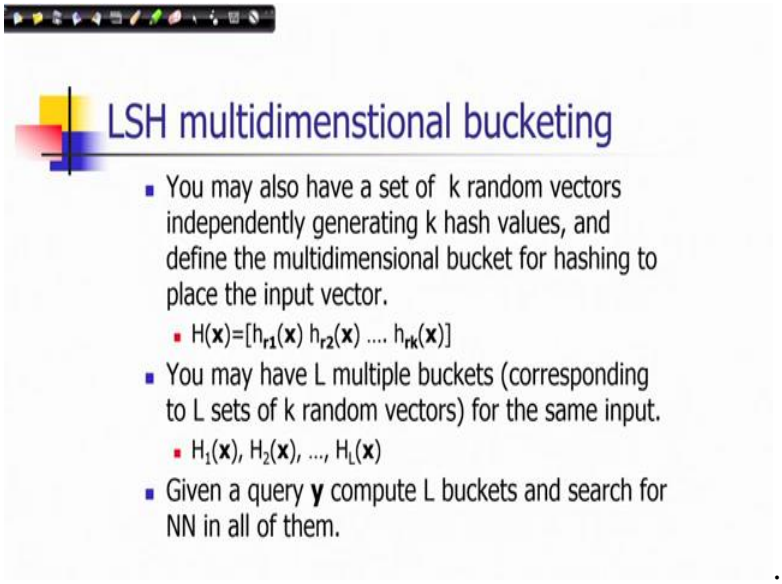
$$h_r(x) = \begin{cases} 1\ if\ x.r > 0 \\ 0\ otherwise \end{cases}$$

So, it is a two valued hash function. So, there are only two vectors either 1 or 0, but the interesting fact is that this would it can be shown that for any two vectors x and y, then probability that two hash values would be equal is given by this fact.

$$\Pr[h_r(x) = h_r(y)] = 1 - \frac{\theta(x,y)}{\pi}$$

So, you can express this angle also in terms of radian, you know that this angle is expressed in terms of radian. In fact, this function is closely similar to cos cosine functions, so which means the angle is very small then those feature vectors are quite nearer and then there is hash values probability of having the same hash value which is equal to 1 in this case. So, any hash value as either both of them are 0 or both of them are 1. So, that would be quite high. So, that is you know particular feature particular property of this or significance of this property.

(Refer Slide Time: 25:29)



So, using this property you can design a scheme. So, you can design a scheme of multi-dimensional bucketing. So, you consider that instead of having a single vector you have a k random vectors and so, independently you can generate k hash values. So, each will give K-dimensional binary representation and that itself can be an index of a bucket, multi dimensional bucket and you can place your input vector in one of these buckets. So, if you have k hash values as if it is a binary representation, there could be 2^K buckets and in one of these buckets.

So, whenever an input vector has the same bucket number, you will be placing them in that bucket. So, in this way you can perform in a hashing. So, this is how a particular bucket is characterized. Now you can further know extend that instead of having a single no single multi-dimensional bucketing scheme, you can repeat this process. You can have L multiple buckets. So, instead of keeping a feature vector only in one bucket, you consider there are L such family of hash functions and independently you are keeping L multiple instances by doing you are making the chance of getting neighboring feature vector, the probability of neighboring feature vector would be high that is a chance by doing it multiple times.

So, you are repeating this process L times and for example, you can generate such you know multi-dimensional vectors hx like L times. So, you have L such buckets and that is the scheme and then when you are performing a nearest neighbor search what you can do

that given a query, you have to compute L buckets and search for nearest neighbor in all of them. So, that makes your locality sensitive hashing more efficient and chance of missing a nearest neighbor will be less.

So, with this let me stop this lecture at this point and we will continue this discussion. In fact, in this topic we have these two parts; one is matching the other part is model fitting which is related as I mentioned because after matching the points, then know using those points you are trying to derive a model which explains the data. So, we will be discussing some of these techniques of model fitting in the continuing lecture.

Thank you very much.

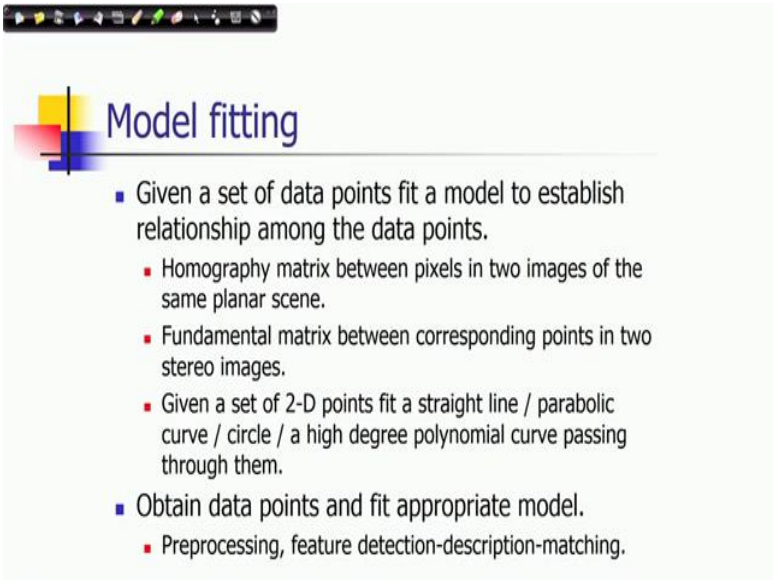
Keywords: K-D tree, nearest search, locality sensitive hashing, bucketing

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture – 31
Feature Matching and Model Fitting Part - III

We are discussing about Feature Matching and Model Fitting. In the last two lecture lectures, we discussed about the techniques for matching feature vectors and also efficient computation of that problem. In this lecture, we will be considering the issue on model fitting.

(Refer Slide Time: 00:35)



So, the computational problem for this fitting model is that given a set of data points we need to fit a model to establish relationship among the data points. For proposing the compression problem first we need to have a set of data points. So, we should obtain data points and then we need to fit an appropriate model to explain those data points. As we mentioned to establish relationship among the data points we need to fit an appropriate model.

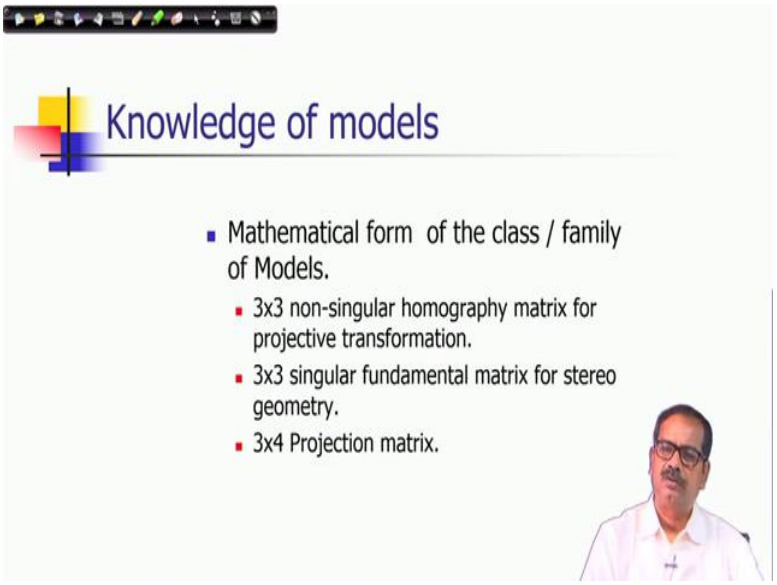
So, in the last few lectures we discussed how we can obtain this data points by detecting feature points and then matching feature points in multiple views in different images, you can establish correspondences and there are various problems on which this correspondences are to be required for fitting a model. Some of the examples could be,

these are also we have discussed in our previous topics. For example, if that between corresponding pixels in two images there exist this homography, then how to compute homography matrix. That itself is a model fitting problem. We have already discussed it solution. So, in this lecture we will consider a bit more general issues involved in this kind of computational problem.

Another example could be again computation of fundamental matrix between corresponding points in two stereo images. Even if I give you a set of 2D points how can you fit a straight line or parabolic curve or circle or a high degree polynomial curve passing through them. There has to be some knowledge about this kind of train or relationships about the model and using that knowledge only you try to get the precise model visual with this points.

For obtaining data points we need to apply various image processing techniques with respect to data points from generated from an image from images, like there are various pre-processing techniques and feature detection description matching that we have already discussed in previous lectures.

(Refer Slide Time: 03:29)



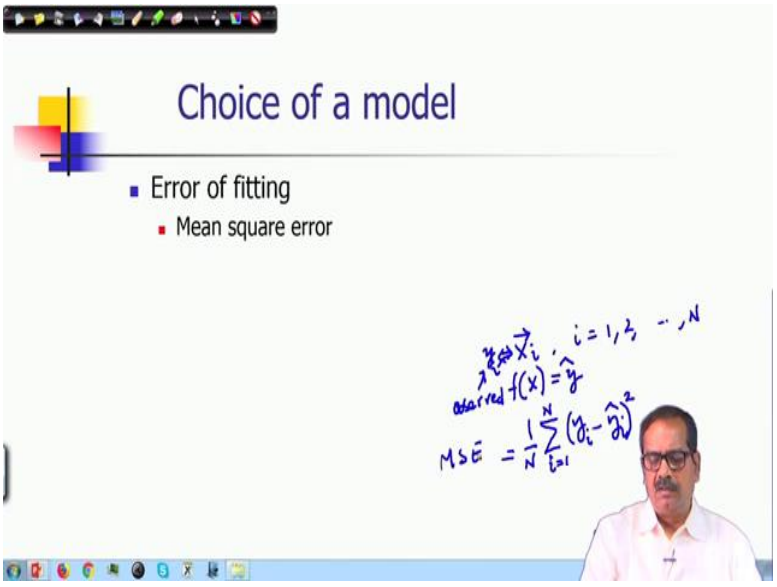
So, as I was referring that knowledge of models is crucial for defining a computational problem. What should be the mathematical form of the class or family of models? For example, we know the mathematical form of a homography relationships among the image points between two sins, between two scenes or images of the same three-dimensional

scene, we know that for a two-dimensional projective transformation the only form of transformation matrix should be in the form of a 3 X 3 non-singular matrix. So, using that knowledge then we have solve that problem that we have already discussed in a previous topic.

Similarly, the form of fundamental matrix in a stereo geometry and its role that has that we know and applying those relationships, we can derive a fundamental matrix from the set of corresponding points. We should have the knowledge of the structure of that fundamental matrix, its properties that it is a singular matrix and also how this matrix relates the corresponding points that request to be used while fitting a model.

Projection matrix of a camera, its form also we have seen how this from could be derived, in the form of a 3 X 4 matrix, where it maps a three dimensional sin point to an image point in the projective space or in the homogeneous coordinate systems. So, finding out or providing a set of corresponding points between three-dimensional sin points and there corresponding image points, we can derive a projection matrix by using this model fitting techniques. This also be discussed in a previous topic.

(Refer Slide Time: 05:53)



So, choice of a model is very important. It comes from the knowledge or analysis of a particular system or analysis how the data are generated, how the data has been obtained, from there you can get that information and you can choose an appropriate model. But there are situations where you may not be able to precisely define a structure of model.

So, you may have to guess even there are also some model some intuitive grace some intelligent grace we can make, but still precisely you do not know how many number of independent parameters that would take, that would play into that model. So, how to ascertain that when you choose a model using some kind of intelligent and intuitive grace work. How that model is appropriate in your data fitting?

So, there are certain checks and bounce for that. Particularly, you need to consider the error of fitting in that case. There are various kinds of errors which were used. We have also used and mean square error between the predicted value and the observed value in our previous model fitting techniques of fitting homography matrices or fundamental matrices or projection matrices. So, similar mean square error could be, similarly we can define mean square error in various other contexts.

Just to explain that what is meant by mean square error though it may be very clear to you through our different exercises, just to elaborate that fact. Suppose, you have some data, let me consider your measurement is a scalar quantity which is y and which depends upon a feature vector. So, corresponding to a feature vector you have a measurement y and you postulated that there exist a functional relationship between the feature vectors and also your measurement that is the model you would consider. You, suppose the form of these functions and apply different techniques to derive this function.

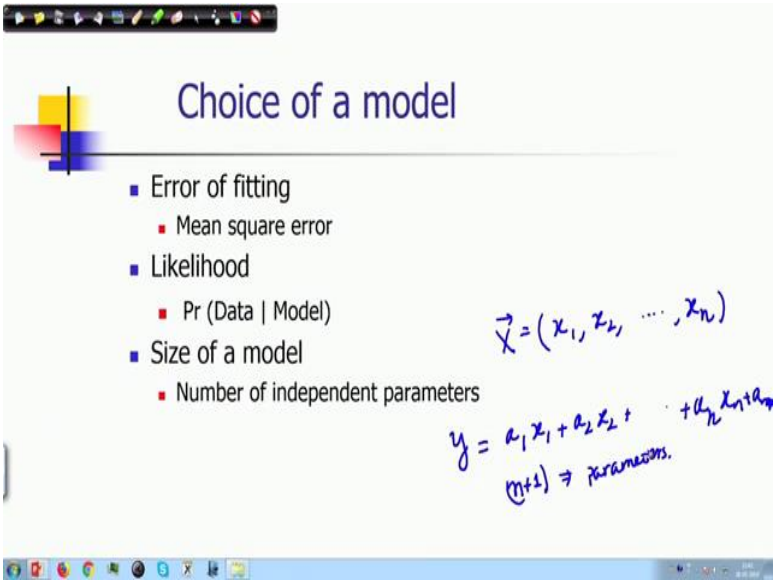
So, now there are when you actually apply a model, then the value what you get that is a model predicted value and this is the value which is the observed value, this is an observed data. So, the error between observation and prediction could be define as square error; the difference if I since I have assumed in this particular case this is a scalar value, so I can simply take the difference. You can extend this concept; when your observation is also a vector and you can consider norm of the differences in that case.

However, so mean square error is that you have so many observation. Suppose, you have n observations and I represent mathematically this pair in this form that means, there are n observations. So, in that case mean square error would be for each observation I will find out the corresponding predicted values which is shown in this form. So, I will sum all these square deviation, square of the errors and take the mean that is what the mean square error is.

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \bar{y}_i)^2$$

In fact, you have used this error in obtaining homography matrix, fundamental matrix or projection matrix in previous computational problems, there mean square error is expressed in terms of norm of a difference vector, mean of the norm of difference vectors. Let us now consider other aspects. This is one particular example of mean square error.

(Refer Slide Time: 11:21)



We can also evaluate the strength of a model by computing likelihood of data given a model, which is defined in this form. What is the probability of occurrence of data given a model? So, this measure should be high, as you know it is a probabilistic measure, so the probability value should lie between 0 to 1 and if it is a very high value your model fitting is good that is one kind of evolution of models. And that determines how good is a model in or when you have a competitive models if we can compute their likelihoods their relative ratios can provide you which model should be consider. There are theories for that, I am just providing in the intuitive reasoning in this case.

Size of a model is important. So, in brief we can say a size of a model is determined by the number of independent parameters those are involved in a model. That is just a very short way of defining a size. If you have more number of independent parameters your module is more complex, that is the rough idea. So, you have to choose a particular size.

Suppose, you have simple linear relationships between the observed between your measurements and also the feature vectors, observed feature vectors and in a feature vector suppose there are n components. So, in that case in a linear model we can write the form of

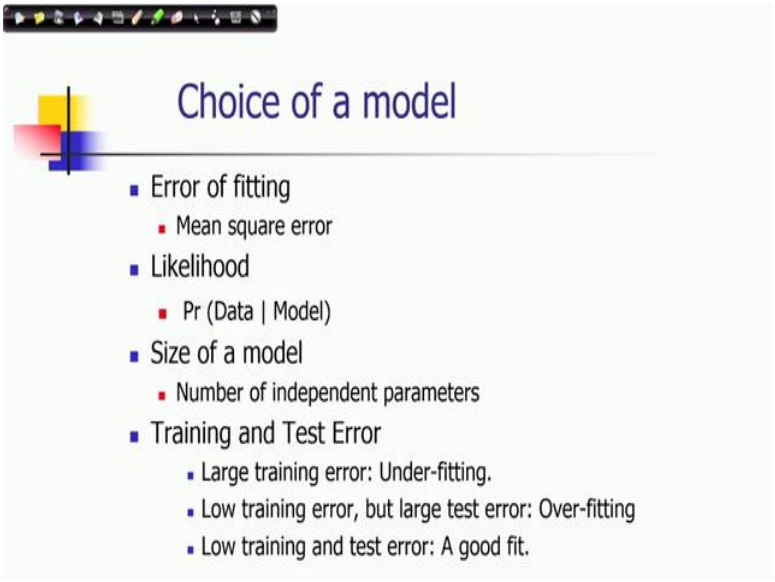
$$y = a_1 x_1 + a_1 x_1 + \cdots + a_n x_n$$

Suppose, your feature vector is n dimensional element, it is represented in an n dimensional space in this fashion and you can also use a constant here, you can use that also in your model. So, we can see in this linear model there are n + 1 parameters. They are the independent parameters.

In this case, you have to establish that whether they are independent or not that depends upon the problem, but let us assume they all independent. So, this is one kind of size of a model. If you consider only a subset of feature vectors, only certain dimensions are related with these observations than your model size also gets reduced. Or, if you want to use the non-linear forms then also there will be coefficients regarding the non-linear terms and your model size will be increasing that as there will be more number of parameters. So, this is some example that depending upon your model description this number of independent parameters they vary and they determine in one way the complexity of a model.

So, the question is what should be the form of a model when as I mentioned that a scenario when you are not sure about the precise structure of the model, as we did in the previous cases of determining of homography matrix, fundamental matrix or projective matrix, the structures were very precisely defined, their properties are also well known or known to us and applying them we have derive them. But in some situations you may not get that then you have to apply intuitive or intelligent gases about this kind of structure, and then observe the error of fitting or likelihood of data based on that model and decide whether you should accept that module or not.

(Refer Slide Time: 16:09)



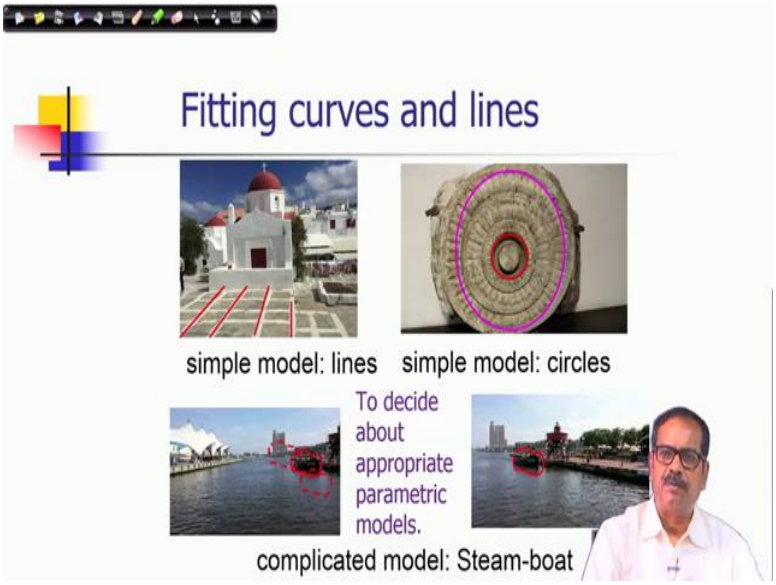
So, regarding this when you are evaluating the performance of a model there are two particular type of errors, those you should note. One is that training error, another is test error. So, training error these term came from the machine learning is perspective. So, when we are fitting a model using a set of data that is the kind of training operations and when you are testing that model with another set of data which is has been kept outside of this training set, but in the test set we know the actual values that means, ground truths values and the we compare those values with the predicted values, then we get also and error while those comparisons that we called test error.

Now, by observing the amount of training error and test error, we can also qualify your model fitting. For example, if your training error is very large that itself reflects that your model is wake it does not explain the data properly, it is an under fitting case which means your number of parameters in the model may have to increased. If it is a low training error that means, in the training you get reasonable you know error term which is very small, but while testing you find actually you are getting large tested which means your model is very data specific and we call that over fitting. It tries to minimize the error just considering that data.

Moment you want to generalize this model over other data sets which are not used in fitting them then actually your model is not performing well. So, this problem is an over fitting case. So, we have to consider that what kind of structure of model you should take. Most

likely you have taken too many parameters, you have to work with less number of parameters in that case. Ideal situation is that you get a low training error and also a test error. Then, that gives you the confidence of having a good fit.

(Refer Slide Time: 18:39)

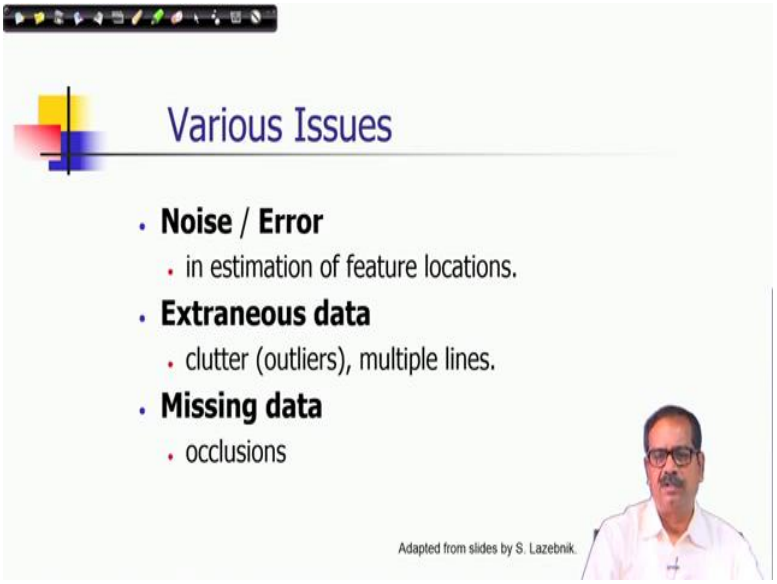


Some examples of model fitting which are observed in image imaging. We have discussed some examples of model fitting regarding homography computations, fundamental matrix computation or projection matrix or camera matrix computation, but from the geometric relationships in a two-dimensional image there are some simple models. As we; I have shown that given the set of points you may have to find out the lines which are which could be formed by this points.

So, the model is that those set of points they lie on a particular line and you are describing that line by their parametric form. It could be a circle or it could be any arbitrary shape consider this scenario and a boundary of an object has been shown in that shape and in another image we would like to see whether that object is present or not. So, using that polygonal models or the boundary described in that kind of shape we are trying to find out whether it exist in other cases or not. But this model is a bit complicated model for any arbitrary shape.

So, the question is that how to decide about appropriate parametric models or appropriate form of a models to relate the data, data points or to establishes relationships among this data points.

(Refer Slide Time: 20:25)



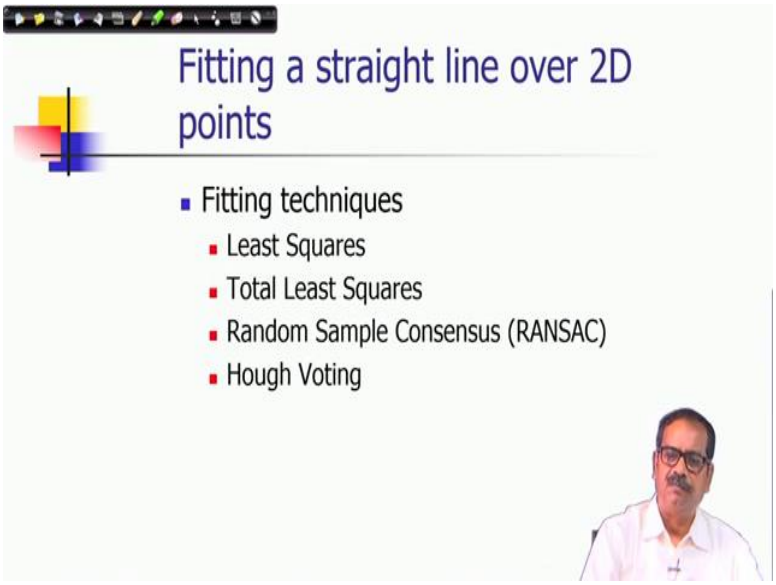
So, there are various issues which are involved in model fitting. Like, your observation could be noisy there could be error. For example, even when you are obtaining data you are applying different computational techniques, different kinds of estimation. So, this estimation will have some error. So, feature locations may not be precisely found, there could be some error in deciding about those locations. There could be some data which is not generated by the process which will fit the model. So, they are called clutter or outliers.

Say, you are assuming the corresponding points between two images there exists an homography and you know that a plane planner, a planner sin induces a homography. So, so assumption is that those points they belong to with a sin points which are lying on the same plane. But it may happen in your observation, in your measurements or in your experiments when you establishes correspondences, when you get those data point some of them may be out of the plane, images of some sin points which are not lying on the same plane. So, if you would like to establish the homography matrix that would provide an error, that would give an erroneous measurements for those models and since you are trying to optimise the overall error that would make a problem.

Then there could be multiple lines. So, your model is a for single line, but actually there are multiple lines and then again the single line model, model for single lines will not be applicable there. Some data could be missing also. For example, some occlusion, so you

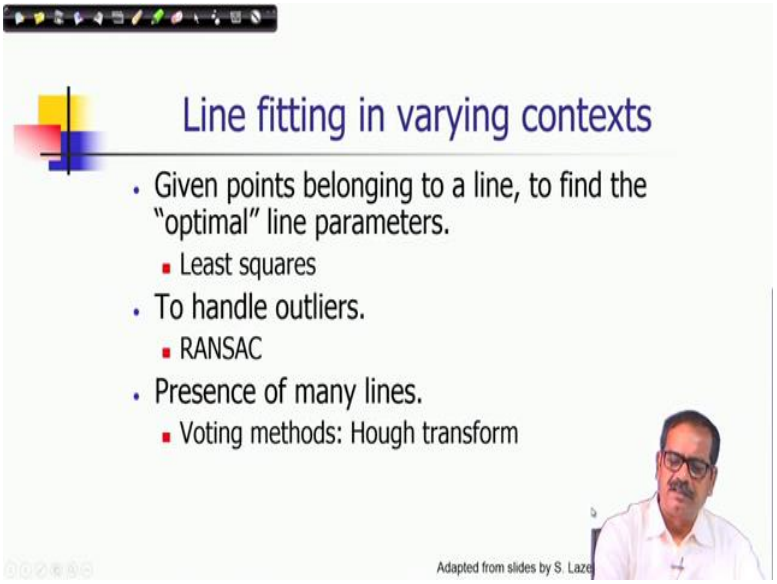
may not get the it is a kind of partial information that you observe in your data points, it owned give you the full picture of the model in that case.

(Refer Slide Time: 22:41)



Now, we will be considering a particular type of problem of model fitting, for this lecture to give you some ideas of you know these issues what we discussed and some approaches to handle them, those could be extended in fitting more complex models also. So, would be considering only a very simple model of fitting a straight line over two-dimensional points and we will discuss about some of this techniques as they are shown here techniques of least squares, total least squares, random sample consensus, then Hough transformed techniques, mentioned here as Hough voting technique.

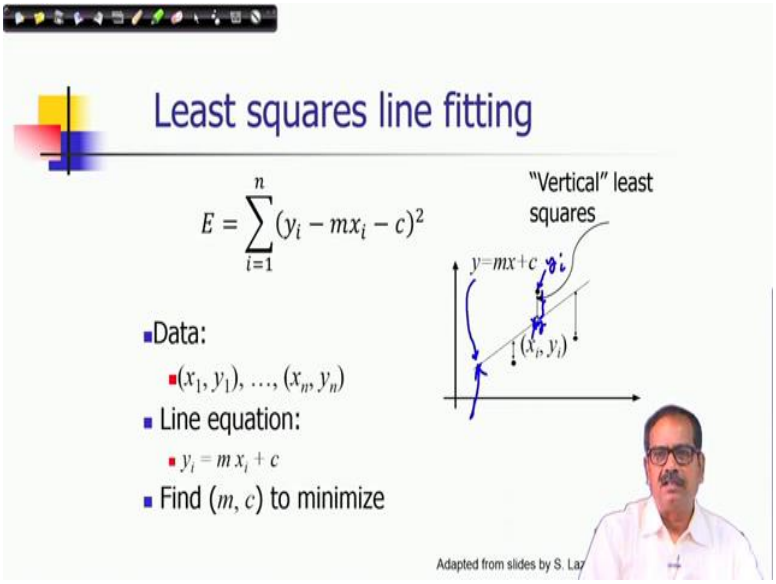
(Refer Slide Time: 23:29)



So, actually these techniques they are presented in varying contexts. Suppose, you have a points belonging to a line and you have to find the optimal line parameters then least squares technique they are applicable, they are very useful, they are very appropriate, but suppose they have outliers in that case we should think about some technique like random sample consensus technique or in short we call it RANSAC. We will see it is a very generic approach of fitting different kinds of models, where we expect there would be outliers in the data or clutter in the data as we mentioned earlier.

If we have too many lines and you would like to fit a model then voting methods are appropriate like Hough transform is applied there. So, in this lecture we will this will be discussing on this different kinds of techniques.

(Refer Slide Time: 24:35)



So, let us first considered the techniques of least squares for line fitting. So, the problem here I have shown that you have a data which is given in the form of a set of two-dimensional points. There are n points and you can see that there coordinates are denoted in this from that means, if you have i-th point its coordinate is denoted in my representation as x i and y i. And the relationship between this coordinates, in the model it is a straight line equation relationship. It is not strictly linear relationship, as you know the straight line is not will not give you a linear relationships it is called a find relationships. Colloquially, even that then we call it a linear fit, but actually it is an a find relationships.

Anyhow, so we know how a line is represented in a two-dimensional coordinate system and that is the familiar representation of y=mx+c that is the model. For a particular instance i-th instance we write it as yi=mxi+c. Then, the error of fit can be expressed in this form,

$$E = \sum_{i=1}^{n} (y_i - mx_i - c)^2$$

So, you can see in this particular diagram, see, if this is the equation or the straight line which has been given by this equation y=mx+c, then given xi say this is your value of y and this is your observed data yi. So, what, the deviation is given y, this vertical shift vertical difference. So, we call it vertical error and square of these deviations will give you mean square error. So, that is how the error term is defined. So, we call it vertical least squares because no we are trying to minimize this error. So, we have to find out that

straight line that m and c which will minimize the sum of square of these errors, which means the error can be expressed in this form and then we need to solve it for this problem. So, find m and c to minimize it.

So, let me stop here and for this lecture. We have understood what problem we need to solve. We will continue this discussion in the next lecture.

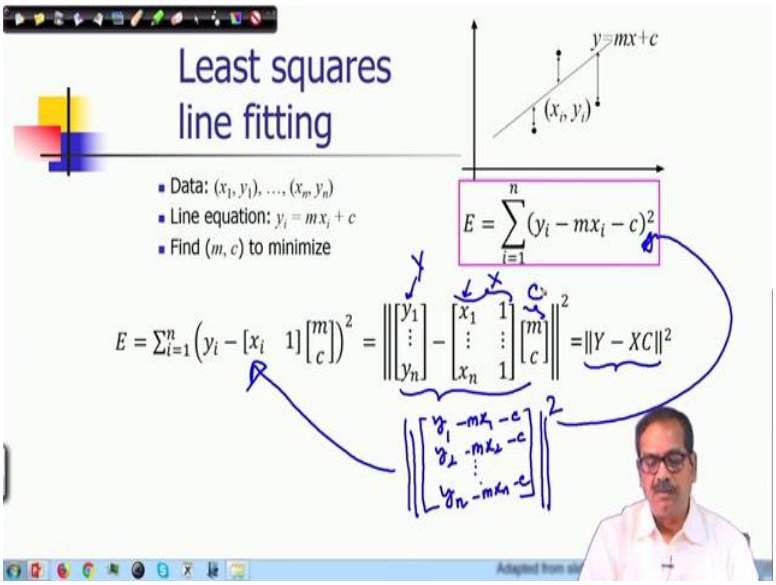
Thank you very much for your attention.

Keywords: Model knowledge, mean square error, fitting curves, occlusions, RANSAC

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture – 32
Feature Matching and Model Fitting Part – IV

(Refer Slide Time: 00:24)



We are discussing about least squares line fitting and as we have considered that we have been given a set of data points and the form of the model in this case is a line, straight line given in the form of equation y=mx+c and the error term is defined as the sum of square of vertical divisions of the observed value from the predicted value, which is been given by the equation

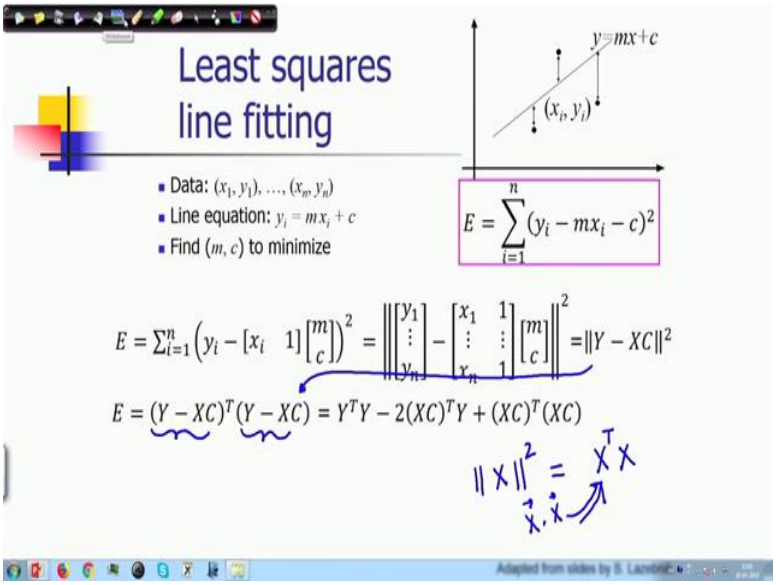
$$E = \sum_{i=1}^{n} (y_i - mx_i - c)^2$$

So, we have to minimize this error with appropriate values of m and c. So, that is a problem, find m and c to minimize this error. So, we can write the expression of error in this form, as you can see here, that we have converted this expression in the short form of matrix notation of your data, just to elaborate that how this representation

$$E = \sum_{i=1}^{n} (y_i - [x_i \; 1][\begin{matrix} m \\ c \end{matrix}])^2 = \left\| \begin{bmatrix} y_1 \\ y_n \end{bmatrix} - \begin{bmatrix} x_1 & 1 \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} \right\|^2 = ||Y - XC||^2$$

Similarly, in the second row we will get y 2 minus mx 2, minus c and in this way, you will get y n minus mx n, minus c. So, this is the column vector and if you would like to take the norm of this vector and take the square of the norm, that exactly we will give you this equation or that is what we will give you also this form, you can verify that. So, this is nothing but, represented in this short form. So, this error E is represented in a shorter matrix notation as capital Y minus XC where, Y is the this matrix, X is this matrix and C is this matrix. So, this is how this notation has been derived, let me wipe out this writings and proceed further.

(Refer Slide Time: 03:05)



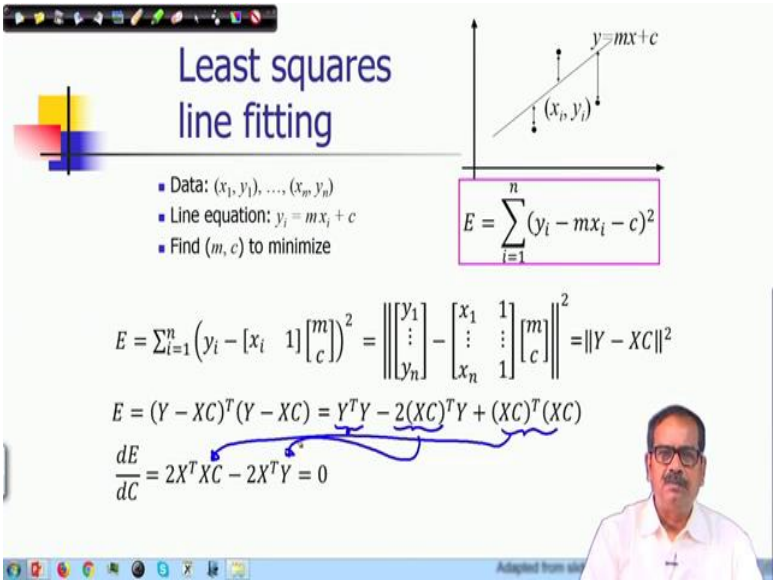
So, where E can be error can be expressed now, we have a matrix notation and as the norm of a matrix can be written in this form

$$E = (Y - XC)^T (Y - XC) = Y^T Y - 2(XC)^T Y + (XC)^T (XC)$$

(Y-XC) just to explain once again suppose, you have a matrix X or a vector, which is represent in the form of a column vector, square of the magnitude, that is nothing but, it is a you understand it is a scalar amount.

And now you expand each one using matrix algebra and then you can perform the multiplications also. Since, matrix is a linear operation. So, you can perform derivatives everything in the matrix form itself and you can extinct the analysis, what we know for ordinary single dimensional variable, single dimensional function cases.

(Refer Slide Time: 04:45)



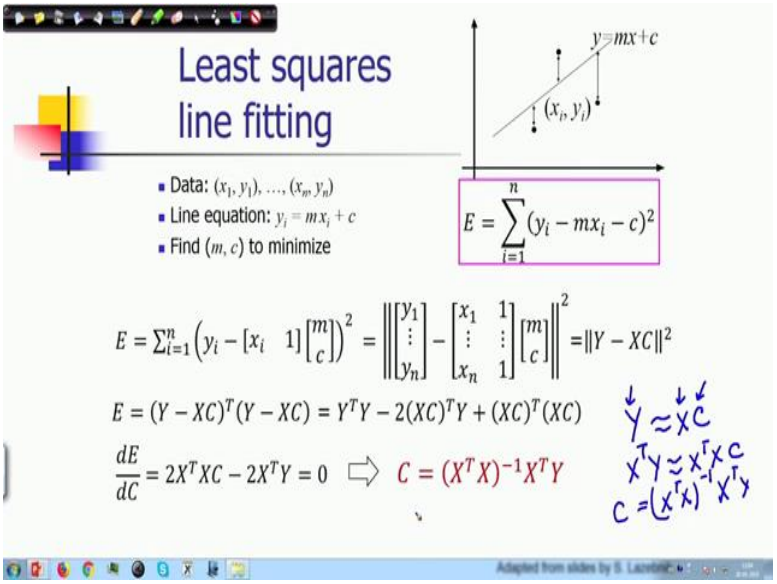
So, in this case, if I perform the derivative with respect to C then, we will get these equations, which means at independently you are taking derivative of E with respect to each component of C,

$$\frac{dE}{dC} = 2X^T XC - 2X^T Y = 0$$

So, this relationships from here, which is established or which can be derived using matrix algebra and their derivative that in short, we have written in this form, as you can see here this is not related to see, this is a kind of constant term, these are the term which are related to C.

You can intuitively extent your knowledge of differential calculus, applying single dimensional, you know variable or applying those or in ordinary single dimensional functional space, you can just extinct them using the matrix notation, you need to know practice on that part, when you are deriving this in a very short and faster way. Otherwise you can do it component wise, we will find they have a one to one relationships with this kind of expression. So, once you obtain this then you perform once again, the algebraic manipulations with this relation.

(Refer Slide Time: 06:40)

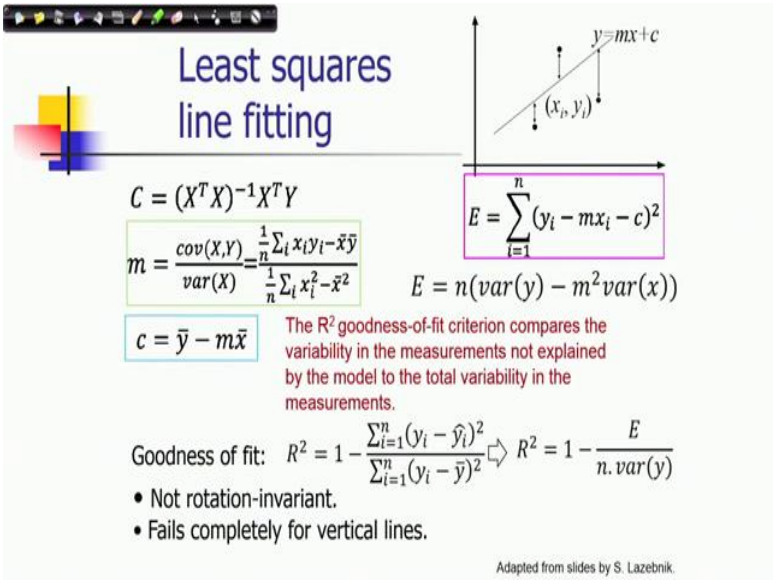


$$C = (X^TX)^{-1}X^TY$$

This is the famous pseudo inverse relationships what we previously also discussed, just to give you the same picture, that I am trying to fit a model like Y=XC. So, I am trying to get C, given Y and X. So, these are the given data.

That is why what exactly we have derived here, it is the result of minimization of the error.

(Refer Slide Time: 07:41)



So, just to expand this relationships in a more granular level of data elements; that means, to get a solution of m and c, we can get this form, m can be expressed as ratio

$$m = \frac{cov(X,Y)}{var(X)} = \frac{\frac{1}{n}\sum_i x_i y_i - \hat{x}\hat{y}}{\frac{1}{n}\sum_i {x_i}^2 - \hat{x}^2}$$

Anyway, there are simpler way of deriving this relationships by performing partial derivative of E with respect to m and with respect to c and following 2 equations and solve for m and c.

So, these are the equations we will get here, y bar and x bar is the mean of x is and mean of. So, y bar is mean of y is and x bar is mean of x is. So, error can be estimated by replacing the value of m and c in that expression, eventually you can find that this would come like this

$$E = n(var(y) - m^2 var(x))$$

So, goodness of fit as I mentioned, that any module fitting you should observe this error, it is related to this error and you can express as a quantity called

$$R^2 = 1 - \frac{\sum_{i=1}^{n} {y_i}^2 - \hat{y}^2}{\sum_{i=1}^{n} {y_i}^2 - \bar{y}^2} \rightarrow R^2 = 1 - \frac{E}{n.var(y)}$$

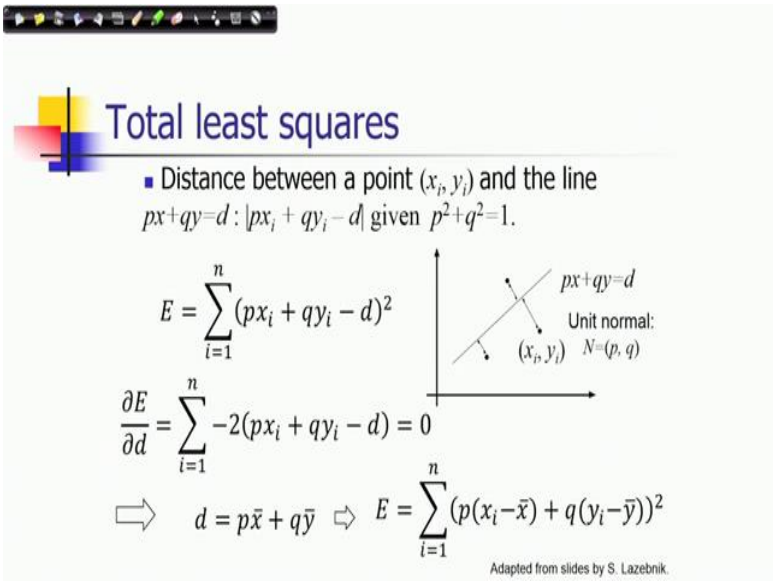
So, it is trying to explain that what part of this variance is explained by this data fitting. So, this is what it compares the variability in the measurements, not explained by the model to the total variability in the measurements.

So, if it is R square value if it is very high, then your model fit is good. So, this value should lie between 0 to 1 and you can also express in this way, this R square is related to E, sometimes it is called coefficients of liberation. So, once you fit a model you should find out also R square and the value should be quite high, say near about 0.8 or more than 0.8 that would be a good fit, we consider that as a good fit in that case.

You should also note that this technique is not rotational invariant and it fails completely for vertical lines because, if it is really a vertical line then, as you can understand your m is going to be almost it is m is. So, if it is vertical line, it is x = C. So, it should be almost

like infinity y is coming now. So, it is very difficult to get term. So, variance of x is training to very very small and m is going to be infinity so, it will fail in that case.

(Refer Slide Time: 11:17)



So, there are techniques which takes care of the situations and also here the objective criteria for fitting model exposed in a different way and this technique is called total least squares. So, it is suitable for any such situation whether, it is a vertical line or not. Now, in this particular diagram it has been shown that, what kind of error measurement we are considering here instead of vertical deviation, rather we are considering the deviation of the point from that line itself. So, deviation means the perpendicular distance between the point and line.

So, those are the deviations and sum of square of those deviations that defined the error, we assume here the line is given in this form. So, px + qy = d. So, that the normal direction of this line is given p q that is the property as you can see. So, the distance between a point and the line as I mentioned, you have to consider the perpendicular distance which is algebraically given in this form,

$$|px_i + qy_i - d| given\ p^2 + q^2 = 1$$

So, you have to consider that representation. That you can make always in your straight line equation by choosing appropriate d or scaling them.

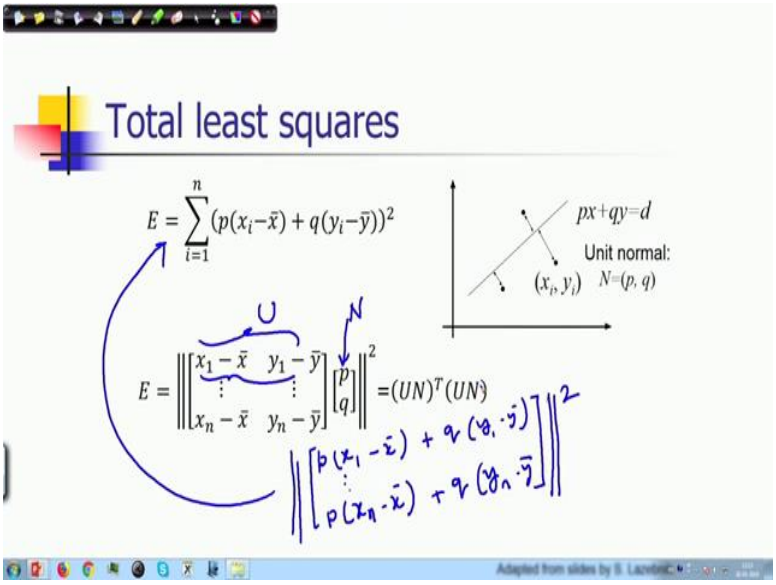
So, the error is given in this form, $px_i + qy_i - d$ is whole square and if I perform the derivative with respect to d then,

$$\frac{dE}{dd} = \sum_{i=1}^{n} -2(px_i + qy_i - d) = 0 \rightarrow d = p\bar{x} + q\bar{y}$$

$$E = \sum_{i=1}^{n} (p(x_i - \bar{x}) + q(y_i - \bar{y}))$$

Now, it is basically a function of p and q by replacing d there and the constant is that p square plus q square should be equal to 1. So, you have to choose that p and q, which will minimize this error E, with the constraint that $p^2 + q^2 = 1$, that is the problem that you need to solve.

(Refer Slide Time: 13:34)



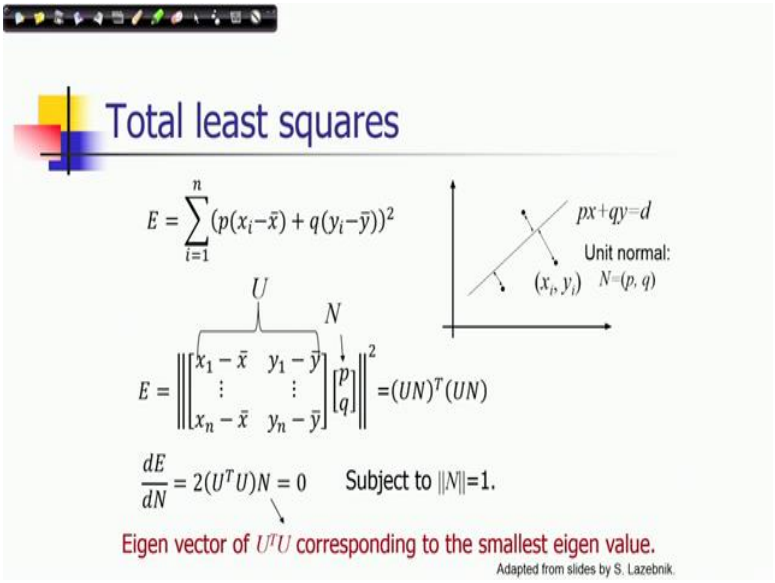
So, once again we can use the matrix notation to represent the square of error, sum of squares in this short form; that means, it is norm of this particular vector. So, just to elaborate once again just that you can see that, if you considered the first row, it is giving you, p into x 1 minus x bar plus, q into y 1 minus y bar and in this way, you consider each row is formed. So, the last row is p x n minus x bar plus q, y n minus y bar. So, this column vector and then if you take it is norm of square. So, you get this expression.

So, this is how it is representing in this form and you would like to make E = 0, you would like to get a solution but, this can be expressed also this is this form is

$$E = (UN)^T(UN)$$

So, this is how this short form is explained and let us continue with this representation.

(Refer Slide Time: 15:16)



So, as I mentioned this is U this is N. So, we need to solve this problem, to minimize this error by choosing appropriate N. So, if I derive E with respect to N, we get this relationships

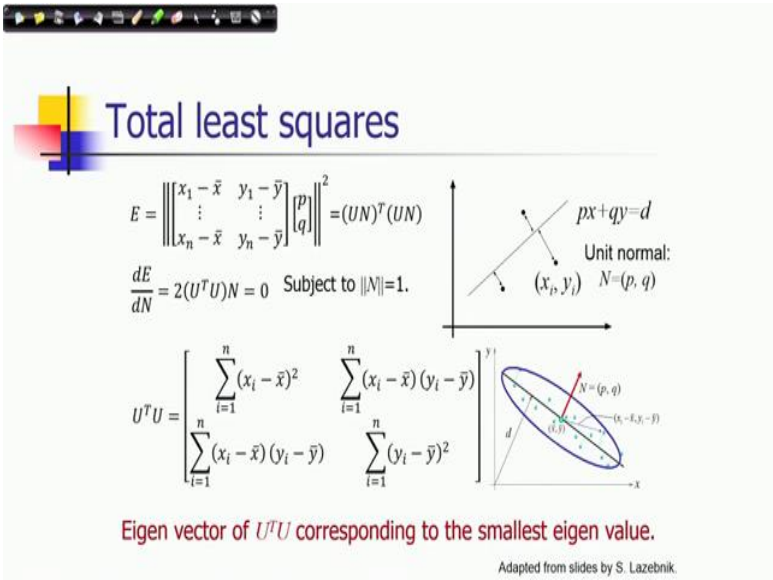
$$\frac{dE}{dN} = 2(U^TU)N = 0 \; subject \; to \; ||N|| = 1$$

which is p and q should be equal to 1 because, that is the constraint we have put, as you can see now, that actually this is giving a set of homogeneous equation.

So, you have to find out the 0 vector of U^TU, you can do it in various ways. So, one of the way could be that if you perform the, if you compute the eigenvectors eigenvalues, there will be one value which should give you the 0 but, ideally it should give 0 but, because of the measurements because of noise etcetera, you should consider that eigenvector which corresponds to the smallest eigenvalue of the matrix U^TU. So, that should be your solution

in this case. So, this is what eigenvector of $U^T U$ corresponding to the smallest eigenvalue, that you should consider.

(Refer Slide Time: 16:53)

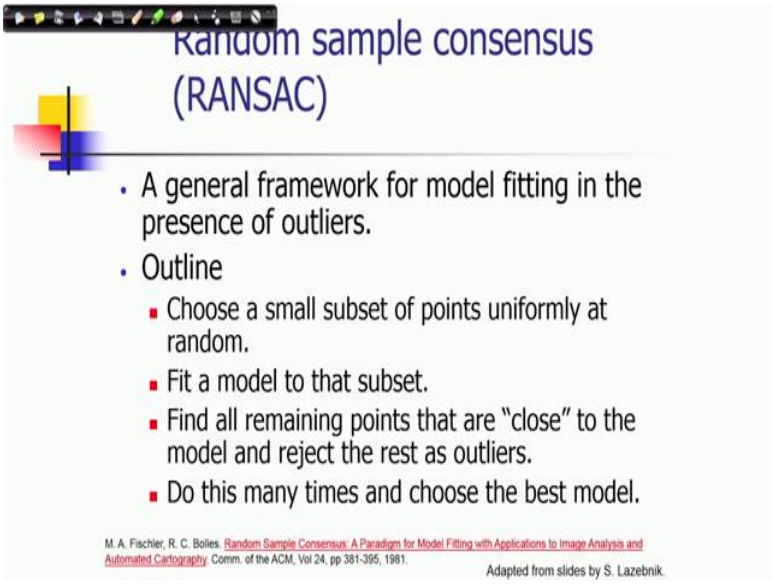


We can understand this result in a more by applying on notion of geometry, as I have summarised the analysis and you can see that the direction of normal is given that N is actually they in a perpendicular direction of the line, what you have fitted here and in the form of px + qy = d, d is an interpretation of parameter d as it is a perpendicular distance from origin to that line.

The structure of $U^T U$ is also interesting to note, you can see that it is a 2 X 2 matrix and whose eigenvalues there will be 2 eigenvalues and you have to choose the minimum eigenvalue and the vectors would be also of dimension 2 X 1 and the diagonal elements and the variances of x coordinates and y coordinates whereas, of diagonal elements and covariance of x and y and this is a symmetric matrix. So, this is what once again eigenvector of $U^T U$ corresponding to the smallest eigenvalue, that is the solution of this problem.

$$U^T U = \begin{bmatrix} \sum_{i=1}^{n} (x_i - \bar{x})^2 & \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) \\ \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y}) & \sum_{i=1}^{n} (y_i - \bar{y})^2 \end{bmatrix}$$

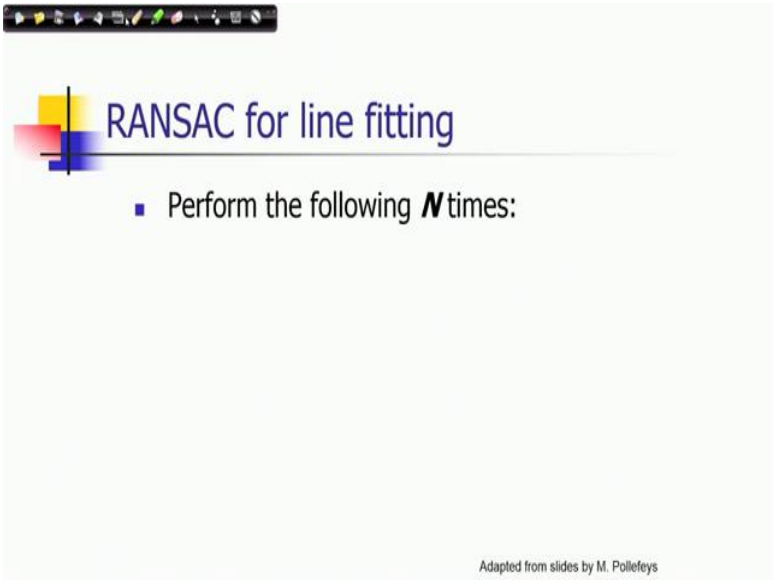
(Refer Slide Time: 18:18)



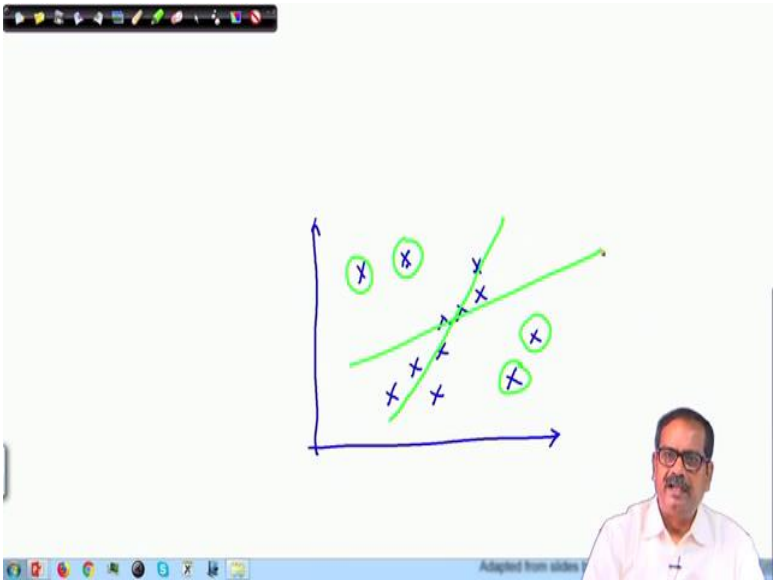
Let us now consider the problem of line fitting, when you have a situation where there are clotted observations or when you there are some outliers in your observations. In fact, this technique though we will be discussing with respect to line fitting, it provides you a general framework for model fitting in the presence outliers. You can apply this technique in our previous cases of finding homograph matrix or fundamental matrix or projection matrix, those we discussed in our previous topics.

So, outline of this technique is that, you should choose a small subset of points uniformly at random and then should fit a model to that subset then, find all remaining points that are close to the model and reject the rest as outliers and do this many times and choose the best model. So, this is what is this technique will be considering and this is what is used or general line of approach will be discussing with respect to a straight line for solving this. So, let me give you a general outline first here let me explain it.

(Refer Slide Time: 19:26)

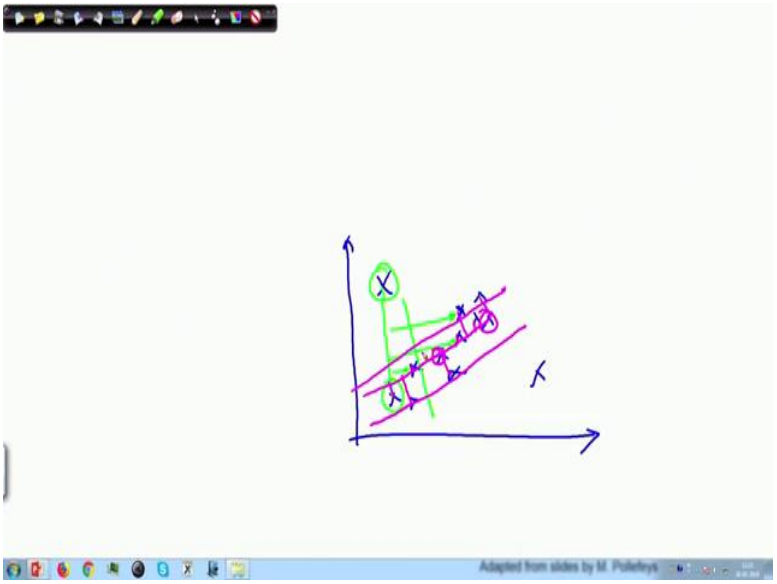


(Refer Slide Time: 19:33)



So, the approach what I mentioned earlier that considered, you have a set of points, which you required to fit in a straight line. There would be some points which are also deviated and which we considered their outline. So, you may consider say this could be a good fit of a straight line among this points but, if I apply the least square error method, the outlier will cause providing you a straight line which is not really fit in this one, it may come something like that, which will minimize the least square error, which is not desirable. So, your objective is to first find out a set of reliable points which we called inlier points and then apply model fitting on those points only.

(Refer Slide Time: 20:35)



So, in that case what we are doing once again, use that examples of some other points, which forms a good inlier points for fitting a straight line and see there are 2 outlier points and as I mentioned. So, what you consider here that you may choose, it may some take some initial setup points, arbitrarily choose any set up points, say initially you have chosen this point and see this point minimally and then you can fit a straight line and then you find out how many points which are lying in vicinity to the straight lines by looking at the perpendicular distance and give a threshold to declare that, they are lying in the outlier, there lying as an inlier point to that model.

Now, if this number is quite high, then you can say that you know this is a good model and then again use all those additional inlier points to refine the fitting of your model. But, as you can see in this scenario, there are only say 2 points, additionally 2 points only 4 points and I mean you can set your parameters in such a way which can decide that this is not a very good fit. So, we will try another setup points.

So, let us consider now that you have chosen, say this point and this point and this is your randomly again you are choosing and this is giving initial setup inlier points and then again you perform this test; that means, you are finding out which pointer line closer to this line and by configuring the distances and you consider all the points, which are outside this lines, that is outside this threshold there outliers.

So, now you can see at least, you can get a good mini points within this straight line and if you are with this number is high, when you may choose this setup points as inlier points and refine your model.

So, unless you get a good setup inlier points, you can go on doing this trial, go on doing this operation N number of times; that means, more number of times and maybe at after certain, if you do not get in any trial there is a good setup inlier points you may drop the idea of fitting this model because, it may happen the data is not good enough for fitting the model otherwise, if you get a good set of inlier points, then you can simply use them and refine your model. So, we will elaborate this process in the next lecture, for the time being let us stop here and.

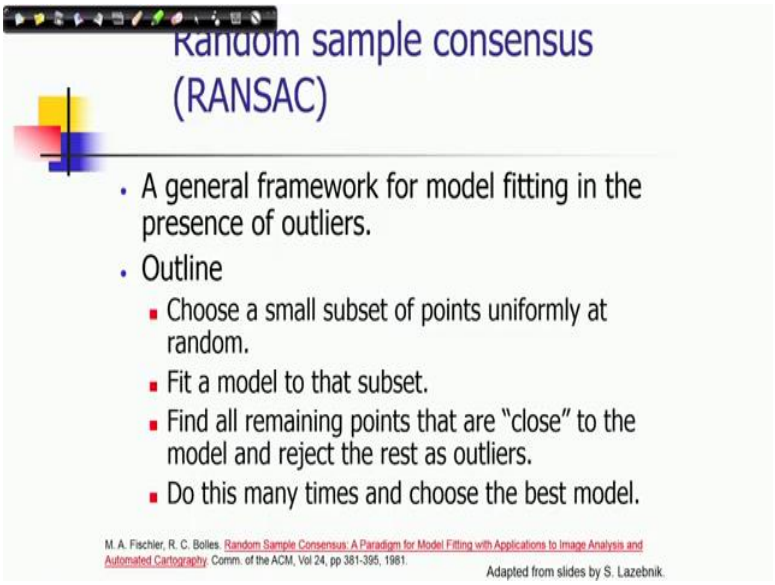
Thank you for your attention.

Keywords: Least squares, line fitting, RANSAC.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

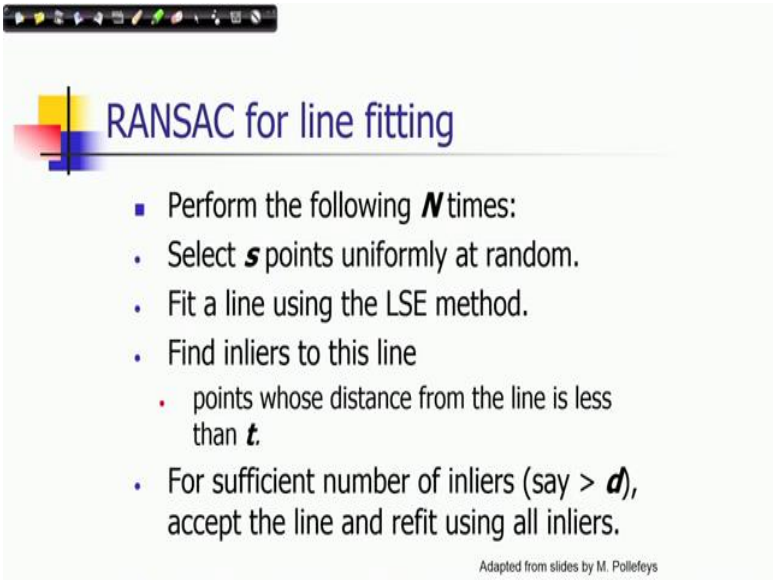
Lecture – 33
Feature Matching and Model Fitting Part – V

(Refer Slide Time: 00:19)



We are discussing about the technique of random sample consensus. And in the previous lecture, I have given a general idea how this technique works while fitting a straight line. Now, we will elaborate it is algorithm with respect to line fitting.

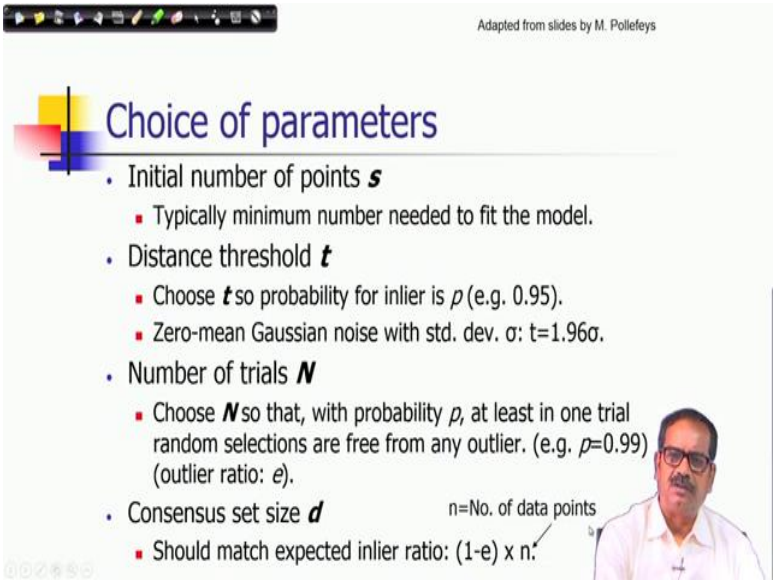
(Refer Slide Time: 00:37)



So, the steps are that as I mentioned that we have to perform N number of times the operations for choosing a good set of inlier points for fitting and initial model. And then based on an initial model and then perform refining over that set of inlier points. So, this is the step that you should select s points uniformly at random.

In the previous example, minimally I have chosen two points for fitting a straight line, but it could be more number of points. And then apply a fitting technique, for example, you can use any least square estimate method which we discussed earlier and then you find inliers to this line, the method is that the points whose distance from the line is less than a threshold value t, then that point is declared is inlier point. And if you have a sufficient number of inlier points, for example, we can have another parameter d, then you should accept the line and then again refine your model by refitting those, refitting the model with those lines with those points only.

(Refer Slide Time: 01:51)



So, there are various parameters those are involved in this process and that will affect your estimation of straight line parameters or fitting the models. So, initial number of points, for example, and as I mentioned you can choose a small number of points, randomly you have to sample those points from your data. Typically, minimum number of points that is require needed to fit the model that at least you have to choose.

So, sometimes you may work on those that minimum number of model sorry, minimum number of sample points and apply the model fit precise model fitting over those sample point. And ones you get a model then you have to compute the distance from that model distance of other points from your straight line, so distance threshold t in this case you should choose this threshold in such a way that probability of that point should be in inlier if it is less than that threshold value that probability should be high.

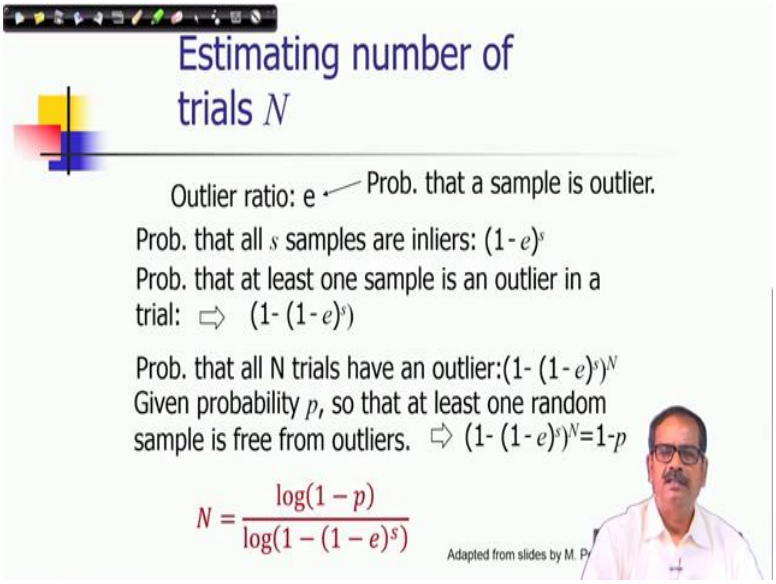
For example, say 0.95 could be your target probability. In that case, if I assume that its say your error in the data is a 0 mean Gaussian noise then you can ensured if its standard deviation is σ then this threshold should be 1.96σ. Then you can ensure that this probability of no getting an inlier with this policy if this 0.95.

Similarly, number of trials that how many times you should repeat these operations for getting a model and then if you do not get any good set of inlier points then you can you then you can terminate the process. So, this number of trials you have to choose in such a way that probability of at least in one trial you should get all the points as inlier that

probability should be very high. Given some data condition, suppose you know that in your data certain fraction of data is there forming outliers and you can have an estimate of that fraction. Say outlier ratio which is given in a symbol e here it could be say 0.1, 0.2 something like that.

And then the consensus set size d that is also important that how many points declared as inliers are sufficient for deciding about a model. So, that also is decided by this outlier ratio. It should match the expected inlier ratio. That means, if there are e fraction of outlier ratios outlier points then there is (1-e)*N number of inlier point is there. So, your d should be close enough to that value, d should not be small, d could be very near to that value. It should not be no significantly less than this expected number. So, these are certain thumb rules by which this parameters are selected as it is shown the N is number of data point.

(Refer Slide Time: 05:12)



So, one particular issue I will be discussing in details here, that how to estimate the number of trials, how it is related with this outlier ratio e and that indicates what the probability that a sample is outlier is. So, I can make probabilistic analysis in this way.

Suppose, we have s samples all of them are inliers. So, in that case probability that all s samples are inliers would be

$$(1-(1-e)^s)^N = 1-p$$

say individual sample is inlier probabilities (1-e) and all s samples together they would be all are inliers would be product of $(1-e)^s$. So, that is what you get here. So, which means that there exist at least one outlier in your set s. So, in a trial that would be $(1-(1-e)^s)$. So, this is what. So, which means a trial, we consider a trial is unsuccessful if there exist at least one outlier that means, I need all the points of sample should be inliers that minimally you should satisfy. So, the probability of in one trial that it is unsuccessful that means, there is an outlier in the set of s points is s shown here.

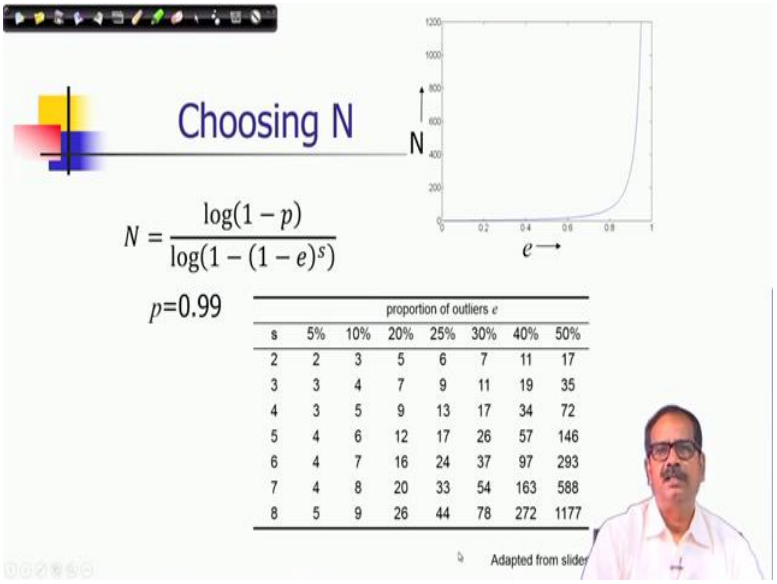
Then for N trails, that none of them contains any successful. It is a product of this N times. So, it is raise this, raise to the power N should give you that probability and 1 minus of that would give you the probability that there is at least one trial where you get all the environments. So, that is what that probability that all N trails have an outlier as I mentioned that value should be raised. So, at least one random trial, one trial is free from any outlier, if I consider that probability p that is the desired feature of your algorithm say p which could be very high value say 0.99, that should be, so your N is related to that value in this fashion.

So, the probability of failing in all N trails is $(1-(1-e)^s) = 1-p$. So, this is how the theoretical estimate of N could be found

$$N = \frac{\log(1-p)}{\log(1-(1-e)^s)}$$

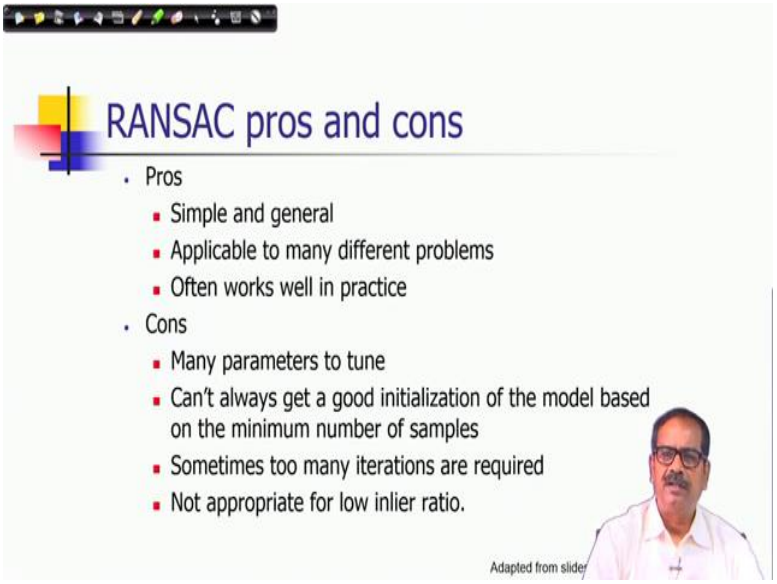
Given the data conditions where outlier issue is given and all other parameters are you know algorithms are also specified.

(Refer Slide Time: 07:59)



If I typically look at those values of N for a very high probability 0.99, we can see that as your samples s varies, the number of trial also varies and if your outlier ratio is very small then you get less number of trials. But if it is large then you get more number of trials, you need to have more number of trials. In fact, it exponentially rises with the rising fraction of outlier layers.

(Refer Slide Time: 08:33)



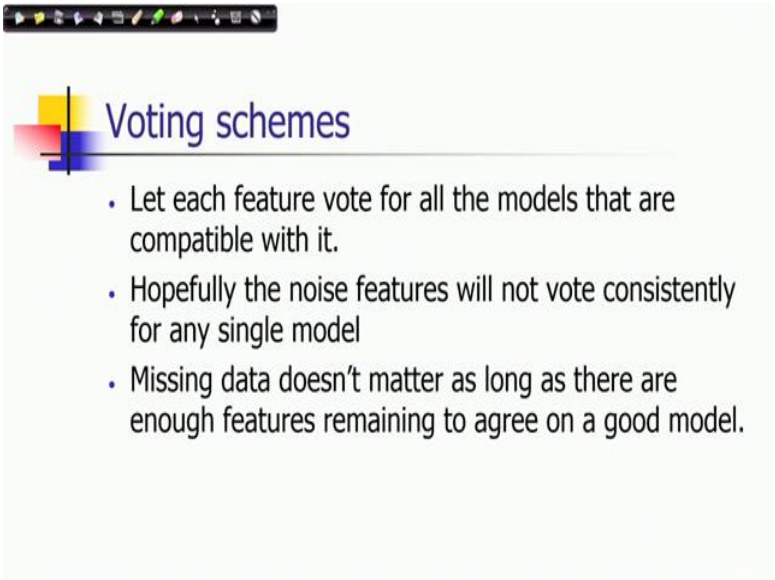
So, there are certain advantages and disadvantages of this RANSAC algorithm. Like, it is very simple and it is general. As you can see that this technique can be extended to other

model estimations. For example, homography matrix or fundamental matrix, there also you when you are selecting a set of data points for fitting a model you should apply this strategy you have choosing a set of inlier data points by rejecting those data points which are not well fitted by the estimated model, initially estimated model.

And then look at the fraction of data points which are validated by model, and then again refitting it with the expanding number of inlier points. So, that is a general approach and that you can apply for any other kind of model fitting which considers now fitting data points. And applicable to many different problems often works well in practice, but there are some of the disadvantages also.

First thing as we have seen there are many parameters to tune. I have already given 4 parameters. So, tuning this parameters as you increase the number of parameters tuning becomes it tricky and non-trivial job, and you may not get a good initialisation of the model based on the minimum number of samples. So, to choose how many samples that s, value of s itself that itself is critical and that influences the performance of the model, performance of this algorithm. Sometimes too many iterations are required that means, you have lot of outliers and your N could be a large number and that is why it is not appropriate for low inlier ratio.

(Refer Slide Time: 10:21)

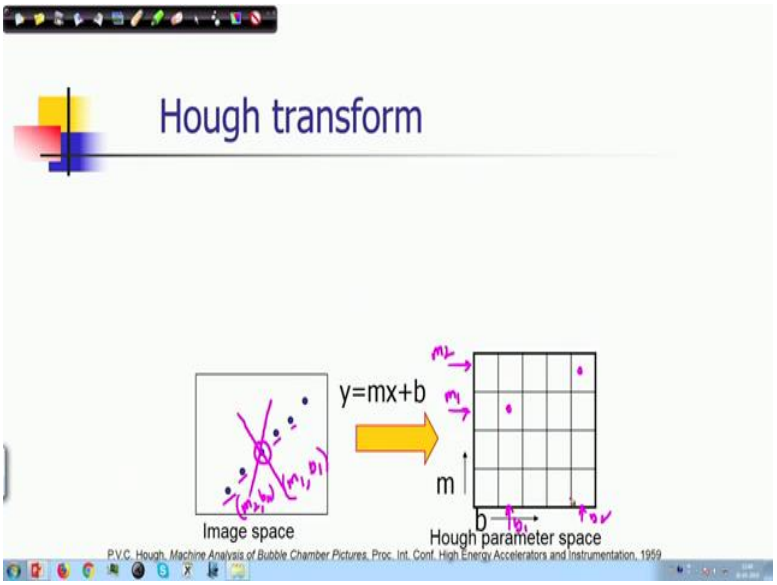


So, let us discuss the other technique flying fitting, where you have multiple lines. And then in this approach will be considering voting schemes for getting a line out of the data

point and in fact, multiple instances of straight multiple instances of similar models out of a data point. In this case, these are all straight line.

So, the idea the general approach of this voting scheme is that let each feature vote for all the models that are compatible with it. So, if the future is noisy, then the consistence consistency of voting owned be there. So, only voting from the less noisy features they would be quite consistent and overall affect would be that the voted, the mostly voted no models they will consistently describe those good set of points. And also in this case, missing data, if it is missing data is also there that can be also handle because no though from the you do not get any vote for the missing data, since you are getting vote from other data and by accumulating this votes you may get a good model.

(Refer Slide Time: 11:40)



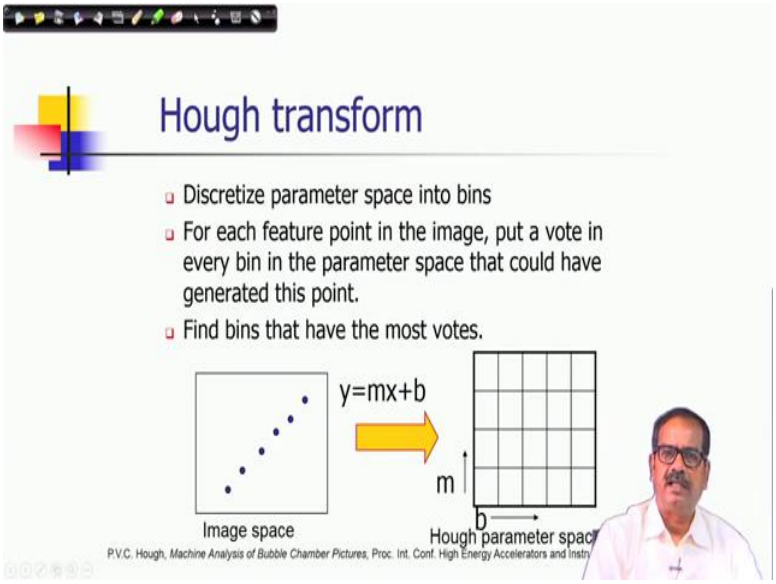
So, let me describe this particular technique with respect to line fitting using Hough transform which follows this approach. You consider a set of points which has been shown here in the image space by circular dots and as we can understand that in fact, there is a straight line passing through these dots.

So, what we are trying to do here that for every point we would like to see that what are the possible straight lines that could pass through that point, which means now if I go to the space of straight lines representations that parametric space of m and b then which pairs of m and b will be describing those straight lines which passes through any point. Just to elaborate that concept consider a particular point say this point.

Now, with respect to this point there are straight lines passing through them say this is described by in the parametric space by (m1, b1), (m2, b2) like this. So, any one parametric space suppose this cell is m1 and say this is b1, and then this cell has a vote for this point. Similarly, say this cell is m2 and say this is b2, so this also has a vote. In this way, in the parametric space you find out all possible combinations of m and b which can provide you those straight lines.

So, we will see mathematically how this could be computed. But this is idea. So, you accumulate votes for all instances of this point sets. You accumulate and you get a distribution of votes in the parametric space. So, the parameter values where these votes are quite high, those are the possible descriptions of a model through this data point. So, this is the idea and that is what we will be considering.

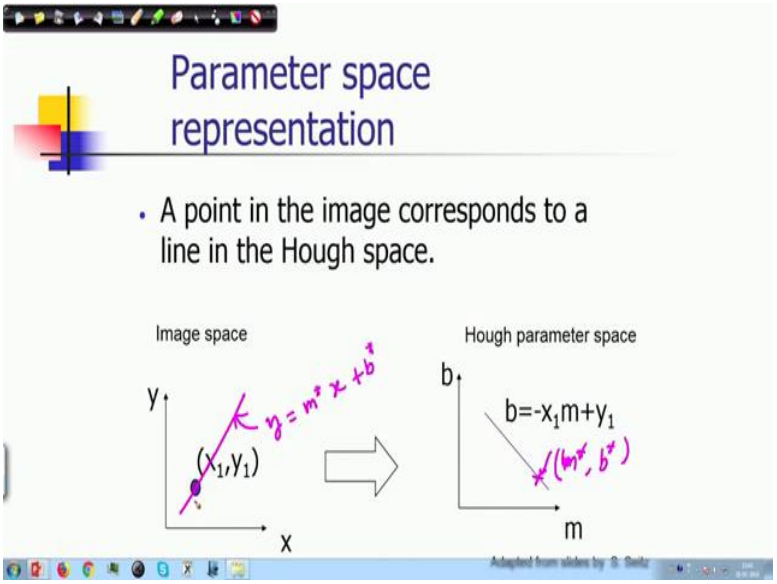
(Refer Slide Time: 13:55)



So, these are the operations which are needed when you perform this particular computations. First thing, we have to discretize the parameter space into bins, because in they continuously are from the continuous domain, but in your computations you have to discretize them, so that you can accumulate votes for each discretized cells.

Then for each feature point in the image put a vote in every bin in the parameter space that could have generated this point. So, that is what I explained here. And we have to find the bins that have the most votes.

(Refer Slide Time: 14:33)

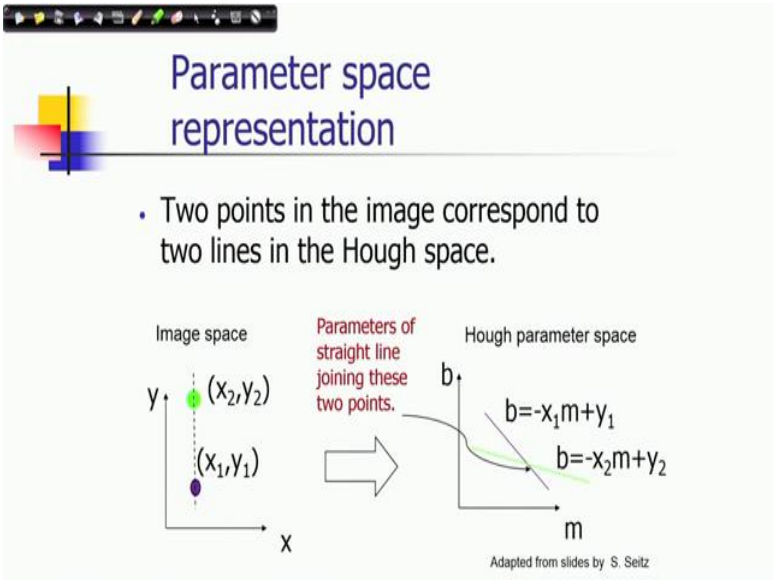


So, this will be explaining further. So, as I was telling that how straight line in the image corresponds to a point in Hough space. So, in the image if you have a straight line as it is shown here y = m1x + b1, you know that exactly it can be represented by a point in the Hough space by (m1, b1). We assume that it is it is corresponding to some discretize bin of m n.

Similarly, if I consider a point in the image, then in the Hough space it will be represented by a line because that is the relationship, say if I consider a line in the Hough space with the parameters b and m expressed in this way b = - x1m + y1, we can see that combination of b and m will give you all lines passing through (x1, y1).

With respect to our previous discussion it means that if I consider any point in this straight line it has a corresponding representation of a straight line in the image space these values, say these values some value corresponds to say m* and b*. And there exist a straight line equation y = m *x + b * which passes through this point (x 1, y1). So, that is an implication of this particular analysis.

(Refer Slide Time: 16:03)

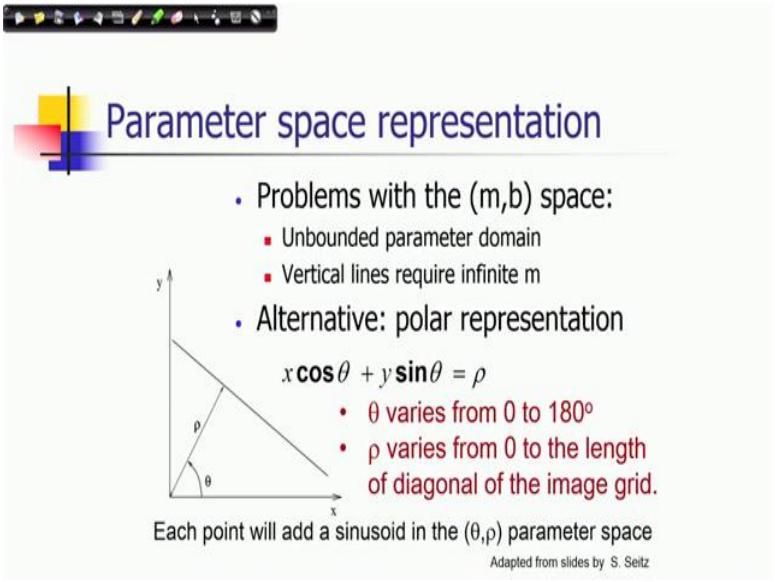


Consider another point (x2, y2) and that would also give you another straight line in the Hough parameter space which has been shown here for the green points I have shown the straight line with green colour which is given by b = - x2m + y2.

So, now you understand the importance of voting's. You can observe that the intersection point of this two straight lines in the Hough parameter space that would be the common parameter of a straight line passing through both the points and which could be a single straight line only geometrically.

Which means, if I use this the intersection of these two straight lines as a parameters of a straight line then that straight line would be joining these two points and that is how the voting comes, voting becomes importance. So, if I collect the votes you will find this intersection point will have more number of votes and since it has more number of votes you may choose that point as a solution. So, this is the general idea.

(Refer Slide Time: 17:05)



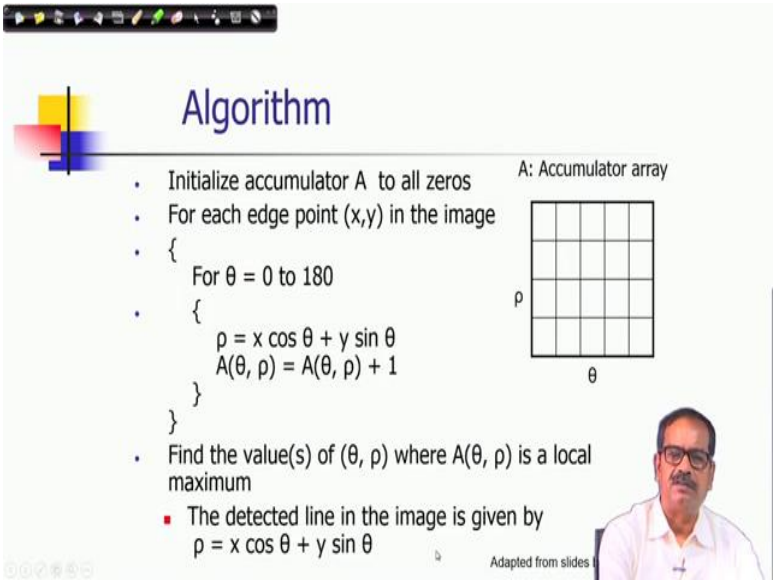
So, there are some problems with this parametric space representation. So, in we are discretizing m and b, because the values of ranges of m and b, they are bit unbounded, so particularly for m. And vertical lines require infinite m there is a problem. So, there are other representations of straight line which is more convenient like polar representation which is given in this form.

So, it is the perpendicular distance from the origin is given by ρ of the straight line and you can write it as

$$xcos\theta + ysin\theta = \rho$$

So, you see here you have parameters θand ρ instead of m and b, and the ranges of θand ρ will be in a finite range. So, like θvaries from 0 to 180 degree for an image and ρ varies from 0 to the length of the diagonal of the image grid. So, we have a very finite range. Discretization becomes easier within this finite range and you can design an algorithm.

(Refer Slide Time: 18:10)

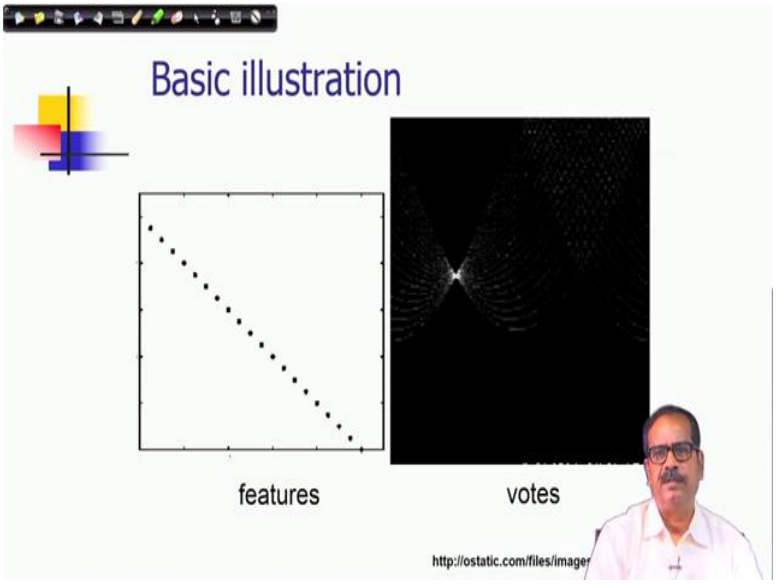


So, the algorithm would be like this that now you consider the discretized space of parameters ρ and θ which is represented by an array here all the cells of an array and let us considered name that array as an accumulated array.

So, you initialize the accumulator a to all 0s, then for each edge point (x, y) in the image you increase the counts of those accumulator cells of those arrays which denote a straight line passing through x y, which means that would be also a straight line which can be given in this form. It is not straight line in rho theta it is given in this form. So, range is theta equal to 0 to 180 degree, then $\rho = x\cos\theta + y\sin\theta$. And in that way the ρ can be found for every θ, then $A(\theta,\rho)$ should be the cell that should be increased value of that it should be accumulated.

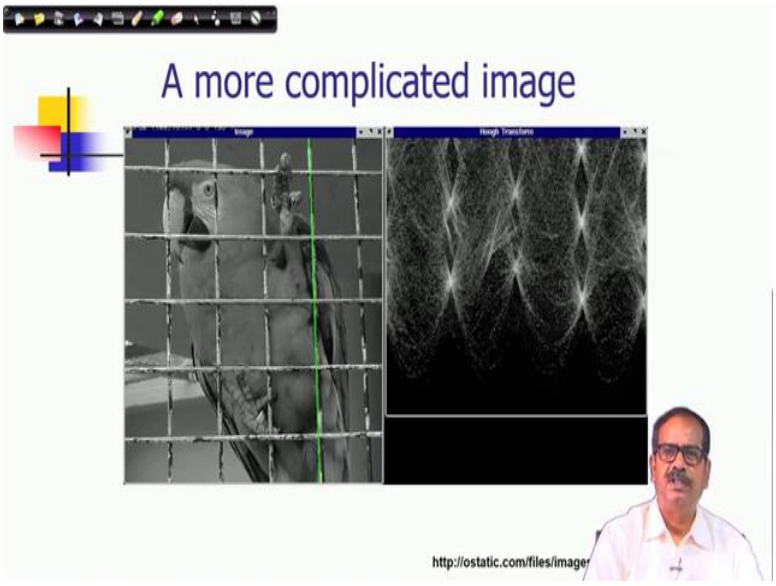
And finally, you need to find the values of θ and ρ which are local maxima because that gives you that shows that there are more votes, there are more votes in those cells considering their surroundings and that is one possible description of a straight line one possible straight line passing through some set of points in your image. And you can, in this way we can get a multiple number of lines, you can detect multiple number of lines those are passing through this set of data point. So, detected line in the image is given by this equation.

(Refer Slide Time: 19:50)



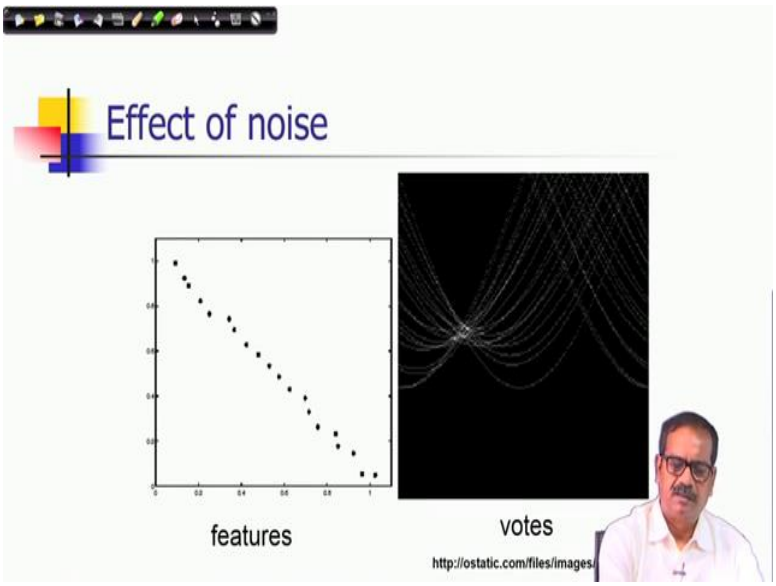
Some illustration that how this votes they look. If you have a very precise straight line passing through all the set of points only single straight line, then you can see there is only one point where you get a very high illuminations. Say, here the brightness value shows the amount of voting and since it is a single line and a very short peak you are expecting in the space of parametric space of Hough transform. So, you will get a very short point there.

(Refer Slide Time: 20:22)



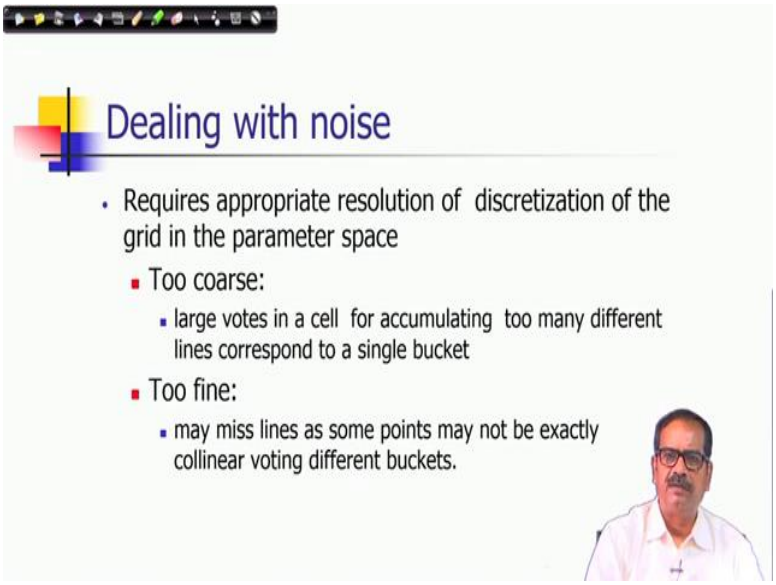
But if you have a more complicated image that means, there are too many number of lines, so each will give you some local maxima our all those voting would be, many voting's would be there from the points lying on their straight line. For example, the line showing on by the colour green would be one of this points in this case.

(Refer Slide Time: 20:45)



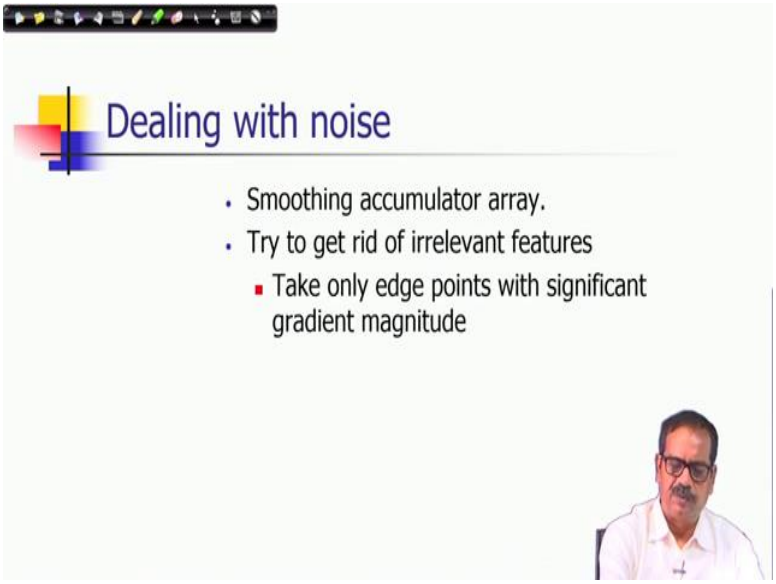
If you have noise then the shortness of that local peak would be lost, would be blurred. See, we can get some near about local maxima. So, there will be cluster of local maxima in a small space, that would be kind of nature of this.

(Refer Slide Time: 21:03)



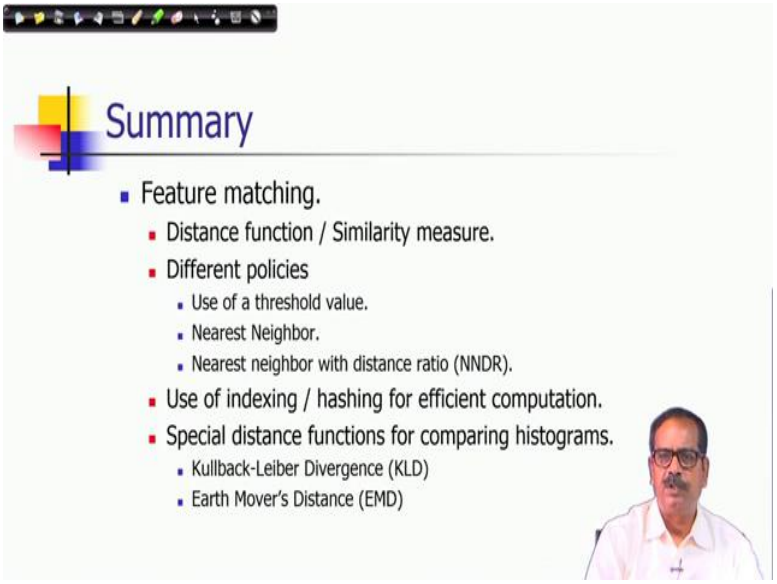
So, these are some situations and one of the major problem or major issue in applying of transform that is it how to deal with noise. In that case, you should require an appropriate resolution of discretization of the grid because if you have two coarse grid in the parametric space then large votes in a cell for accumulating too many different lines correspond to a single bucket. If it is too fine, then you a miss lines as some points may not be exactly collinear and voting different buckets.

(Refer Slide Time: 21:38)



So, you should have an appropriate resolution, and other than this you should smooth the accumulator array to reduce the effect of ripples, effect of rippling local maximas and sometimes you try to get rid of irrelevant features that means, the outliers which may cause problem. So, you fit and you take the edge point with significant. So, in this case, you consider those points which have a strong gradient magnitude and use them for your fitting.

(Refer Slide Time: 22:13)

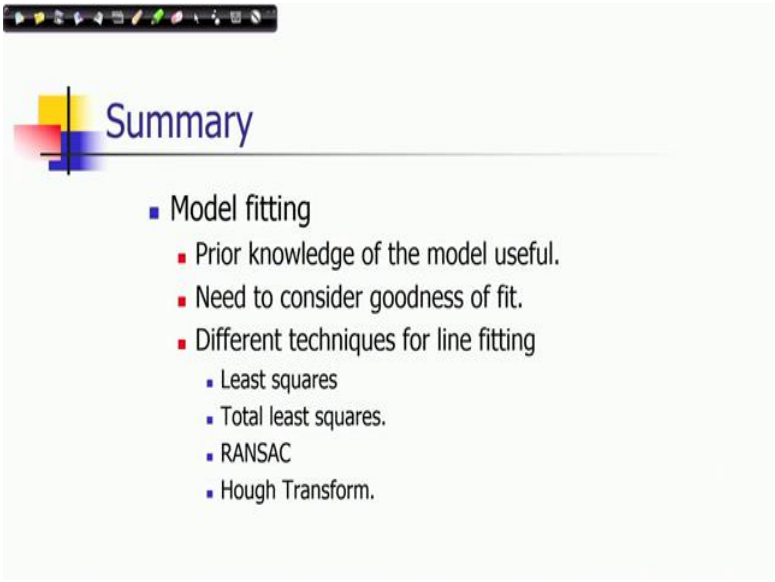


So, here we come to the end of this particular topic and let me summarise what we discussed in this topic of feature matching and model fitting. So, in feature matching we have discussed about different distance functions and similarity measure, and then we also considered different policies of matching like you can use a fix threshold value to report the matching points and within that neighbourhood of a point. Then, you can also choose in precise nearest neighbour that is as a matching point matching feature or you can use nearest neighbour with distance ratio that policy used from the query point you can find out, you can use this measure to compute a matching feature point.

The indexing and hashing to make your efficient your computation efficient and in particular with respect to these feature vectors you can exploit the geometry and you can use the indexing schemes like K-D tree or for hashing locality sensitive hashing schemes that we discussed. For comparing histograms we discussed about different some of the special distance functions other than using the norms, Lp norms with the histogram, correspondingly histogram bins values. We can use the special measures like Kullback Leiber divergence assuming histogram is a probability density function. And then, this measure is used to compare between compare two distributions and if there are close this value should be less.

And earth mover's distance also we discussed, that is a special distance function and its computation is also quite involved computations.

(Refer Slide Time: 24:14)



In the model fitting approaches, we considered only of course, here the issues of fitting straight lines given two-dimensional points, but we discussed some of the general issues a model fitting like prior knowledge of the model is useful, then we should consider the goodness of fit when you fit a model. And we discussed about different techniques for line fitting, like techniques of least squares, then total least squares, and the random sampling consensus technique or RANSAC, and also Hough transform technique. So, with this, let me conclude this lecture and this topic. In our next topic would be on colour processing.

Thank you very much for your attention.

Keywords: RANSAC, outliers, hough transform, parametric space.